

Technical University of Munich

School of Natural Sciences

Master's Thesis in Quantum Science & Technology

Laurin Demmler, B.Sc.



Technical University of Munich

School of Natural Sciences

Master's Thesis in Quantum Science & Technology

A Nested Amplitude Amplification Protocol for the Binary Knapsack Problem

Ein geschachteltes Amplitudenverstärkungsprotokoll für das binäre Knapsackproblem

Author:	Laurin Demmler, B.Sc.
Examiner:	Christian Mendl, Prof. Dr.
Supervisor:	Maximilian Hess, Dr.
Start Date:	17.11.2025
Submission Date:	28.09.2026

I confirm that this Master's thesis is my own work and I have documented all sources and material used.

Munich, 28.09.2026 Laurin Demmler, B.Sc.

Transparency in the Use of AI Tools

In preparing this thesis, Claude Opus 4.5 as well as Claude Opus 4.8 were used. Both models were employed to assist in literature review, to ensure grammatical and semantic correctness, to help with styling guidelines as well as to support in creating figures. The codebase has been refactored using Claude Opus 4.8. Documentation as well as simulation wrapping and plotting functions have been produced with the help of AI.

Acknowledgements

I would like to thank Dr. Maximilian Hess for offering guidance, help and for being a friend to count on. I would also like to thank Hans Ehm and Prof. Christian Mendl for the joint supervision and freedom to explore the topic of this thesis.

Abstract

The binary knapsack problem is an NP-hard combinatorial optimization problem that arises as a subproblem in semiconductor supply chains. Grover Adaptive Search (GAS) tackles such problems by iteratively applying Grover search to find solutions that improve upon an incumbent. The Quantum Tree Generator (QTG) can be used to warm-start GAS for the knapsack problem by creating a (biased) superposition of feasible states. In this thesis, we design and evaluate the Nested Amplitude Amplification (NAA), which utilizes an additional amplitude amplification subroutine over the first s of the n items of the knapsack problem, before performing GAS on all decision variables. We expect this nesting to be advantageous because amplifying a promising partial state concentrates amplitude on assignments that can be completed to high-quality solutions, so that the GAS requires fewer Grover iterations. We propose two criteria that mark partial assignments, the profit-based criterion and the profit-and-capacity-based criterion, and analyse the gate count requirements of the NAA in depth. Against a baseline consisting of a QTG warm-start and GAS, we evaluate the NAA by the optimality gap reached under a fixed gate count budget, using a simulation based on classical amplitude tracking that reduces the computational cost compared to statevector simulations. Results are computed for uncorrelated and weakly correlated knapsack instances of size up to $n = 35$. Using a non-zero bias parameter in the QTG, the nested approach does not outperform the baseline in any investigated regime, struggling to efficiently predict a favourable depth s, and degenerating into an amplitude amplification-free warm-start. Without bias in the QTG, the amplification yields a visible advantage in specific capacity regimes, although against a baseline known to be far from its optimum.

Zusammenfassung

Das binäre Knapsackproblem ist ein NP-schweres kombinatorisches Optimierungsproblem, das als Teilproblem in Halbleiter-Lieferketten auftritt. Grover Adaptive Search (GAS) geht derartige Probleme an, indem es iterativ Grover's Algorithmus anwendet, um Lösungen zu finden, die die aktuell beste Lösung verbessern. Der Quantum Tree Generator (QTG) kann GAS für das Knapsackproblem warmstarten, indem er eine (nicht-uniforme) Superposition zulässiger Zustände erzeugt. In dieser Arbeit entwerfen und evaluieren wir die Nested Amplitude Amplification (NAA), die eine zusätzliche Amplitudenverstärkung auf die ersten s der n Gegenstände des Knapsackproblems ausführt, bevor wir GAS auf alle Entscheidungsvariablen anwenden. Wir erwarten, dass diese Verschachtelung vorteilhaft ist, weil die Verstärkung eines vielversprechenden Teilzustands Amplitude auf Zuständen konzentriert, die sich zu hochwertigen Lösungen vervollständigen lassen. In der Folge könnte GAS weniger Grover-Iterationen benötigen. Wir schlagen zwei Kriterien vor, die partielle Zustände markieren: das profit-based criterion und das profit-and-capacity-based criterion, und analysieren den Gatterbedarf der NAA im Detail. Gegenüber einer Baseline aus QTG-Warmstart und GAS bewerten wir die NAA anhand der optimality gap, die unter einem festen Gatterbudget erreicht wird, mittels einer Simulation auf Basis von klassischem Amplitude Tracking, die den Rechenaufwand gegenüber Zustandsvektorsimulationen reduziert. Die Ergebnisse werden für unkorrelierte und schwach korrelierte Knapsackprobleme mit einer Größe von bis zu $n = 35$ berechnet. Mit einem von null verschiedenen Bias-Parameter im QTG übertrifft der verschachtelte Ansatz die Baseline in keinem untersuchten Bereich, wobei er sich mit der effizienten Vorhersage einer günstigen Tiefe s schwertut und zu einem Warmstart ohne Amplitudenverstärkung degeneriert. Ohne Bias im QTG erzielt die Verstärkung in bestimmten Kapazitätsbereichen einen sichtbaren Vorteil, allerdings gegenüber einer Baseline, die weit von ihrem Optimum entfernt ist.

Contents

1 Introduction 1

2 Background 4

2.1 The Knapsack Problem 4

2.2 Grover's Algorithm 5

2.3 Amplitude Amplification (AA) 7

2.4 Grover Adaptive Search (GAS) 7

2.5 Quantum Arithmetic Circuits 10

2.5.1 Quantum Fourier Transform (QFT) Adders 10

2.5.2 Integer Comparison 12

2.6 Quantum Tree Generator (QTG) 14

2.6.1 QTG as a Quantum Circuit 14

2.6.2 QTG as a Classical State Sieve 17

2.6.3 Gate Count for the QTG 18

2.6.4 A note on bias 20

3 Nested Amplitude Amplification (NAA) Algorithm Design 22

3.1 Motivation 22

3.2 Baseline Approach (BA) 22

3.2.1 Implementation 22

3.2.2 Gate Count 24

3.3 General Nested Amplitude Amplification 24

3.4 Gate Count of the Nested Approach 28

3.5 Nested Amplification Criteria 30

3.5.1 Profit-based Criterion (PBC) 30

3.5.2 Profit-and-Capacity-based Criterion (PCBC) 31

3.6 Termination Criteria 35

3.6.1 t_{IIF} 36

3.6.2 t_{GAS} 37

4 Implementation and Simulation Approach 39

4.1 Instance Generator 39

4.2 Classical Amplitude Tracking 41
4.3 Experimental Metrics 45
4.3.1 Gate Count Ratio 45
4.3.2 Optimality Gap 45
4.4 Implementation Pipeline 46
5 Results 48
5.1 Average Hamming Weight 48
5.2 Bias Sweep 49
5.2.1 Bias Sweep for the Profit-based Criterion (PBC) 50
5.2.2 Bias Sweep for the Profit-and-Capacity-based Criterion (PCBC) 53
5.3 Inner Iterations 55
5.3.1 Inner Iteration Behavior for the Profit-based Criterion (PBC) 56
5.3.2 Inner Iteration Behavior for the Profit-and-Capacity-based Criterion (PCBC) 57
5.3.3 Interpretation of the Results 58
5.4 Runtime Depth Prediction 58
5.4.1 Depth Fraction Sweep 59
5.4.2 Depth Schedule 61
5.5 Fixed Budget Performance 62
5.6 Unbiased Fixed Budget Performance 64
6 Conclusion 69
Bibliography 72

List of Figures

Figure 1 Flowchart of the Grover Adaptive Search (GAS) algorithm. 8

Figure 2 Quantum circuit of the Quantum Tree Generator (QTG) for n decision variables. 17

Figure 3 Quantum circuit of the Baseline Approach (BA) for one Amplitude Amplification (AA) step of Grover Adaptive Search (GAS). 23

Figure 4 State preparation circuit A_s^{PCBC} for the Profit-and-Capacity-based Criterion (PCBC) at depth s. 34

Figure 5 Implementation pipeline for benchmarking Nested Amplitude Amplification (NAA) against Baseline Approach (BA). 47

Figure 6 Hamming distance $\Delta\big(x^{\mathrm{opt}}, x^{\mathrm{greedy}}\big)$ vs. number of items n. 49

Figure 7 Profit-based Criterion (PBC) bias sweep on uncorrelated instances. 51

Figure 8 Profit-based Criterion (PBC) bias sweep on weakly correlated instances. 52

Figure 9 Profit-and-Capacity-based Criterion (PCBC) bias sweep on uncorrelated instances. 53

Figure 10 Profit-and-Capacity-based Criterion (PCBC) bias sweep on weakly correlated instances. 54

Figure 11 Gate count ratio distributions for the Profit-based Criterion (PBC) by inner-iteration count. 56

Figure 12 Gate count ratio distributions for the Profit-and-Capacity-based Criterion (PCBC) by inner-iteration count. 57

Figure 13 Gate count ratio vs. depth fraction for the Profit-based Criterion (PBC). 59

Figure 14 Gate count ratio vs. depth fraction for the Profit-and-Capacity-based Criterion (PCBC). 60

Figure 15 Best depth fraction distribution for the Profit-based Criterion (PBC). 60

Figure 16 Best depth fraction distribution for the Profit-and-Capacity-based Criterion (PCBC). 61

Figure 17 Cumulative candidate coverage of the depth schedule for the Profit-based Criterion (PBC). 62

Figure 18 Cumulative candidate coverage of the depth schedule for the Profit-and-Capacity-based Criterion (PCBC). 62

Figure 19 Biased Fixed-budget performance by instance class. 63

Figure 20 Capacity-to-weight ratio sweep for unbiased Nested Amplitude Amplification (NAA) performance by instance class. 65
Figure 21 Unbiased fixed-budget performance in selected capacity-to-weight regimes. 66
Figure 22 Unbiased size scaling in selected capacity-to-weight regimes. 66

List of Algorithms

Algorithm 1 QSearch 9
Algorithm 2 Grover Adaptive Search 9
Algorithm 3 Quantum Tree Generator 16
Algorithm 4 Classical Tree Generator (CTG) 18
Algorithm 5 Inner Iteration Finder (IIF) 26
Algorithm 6 Nested Amplitude Amplification 27

List of Tables

Table 1 Clopper-Pearson bounds for $L = 5$ Inner Iteration Finder (IIF) measurements. 37

List of Abbreviations

AA – Amplitude Amplification

BA – Baseline Approach

BnB – Branch-and-Bound

CNOT – Controlled NOT

CTG – Classical Tree Generator

CWR – Capacity to Weight Ratio

GAS – Grover Adaptive Search

IIF – Inner Iteration Finder

MPS – Matrix Product States

NAA – Nested Amplitude Amplification

NP – Nondeterministic Polynomial time

PBC – Profit-based Criterion

PCBC – Profit-and-Capacity-based Criterion

QAOA – Quantum Approximate Optimization Algorithm

QFT – Quantum Fourier Transform

QTG – Quantum Tree Generator

UB – Upper Bound

VQE – Variational Quantum Eigensolver

1 Introduction

Managing and optimizing semiconductor supply chains and production is a highly non-trivial task [1]. Many optimization problems that arise in this context belong to the class of Nondeterministic Polynomial time (NP)-hard problems and are of a scale that makes solving them to optimality computationally expensive [2]. Particularly relevant subproblems in semiconductor supply chains include multi-commodity flow, scheduling and knapsack problems [3–5]. In this thesis, we focus on the latter, because of its relevance, relative simplicity[1] and NP-hardness (for more details, refer to Section 2.1).

In a classical optimization context, the knapsack problem has been studied thoroughly [6–8]. Although NP-hard in the worst case, many instances are solvable efficiently in practice, and only specific structured regimes remain computationally challenging [9]. Some approaches are focused on obtaining fast, but potentially sub-optimal solutions [10], whereas other algorithms aim to find the optimal solution (including an optimality certificate) in exponential time [11]. A particularly interesting approach is the Branch-and-Bound (BnB) algorithm [11], which discards large parts of the solution space by excluding partial knapsack solutions that cannot be extended to an optimal solution. This thesis, although mainly concerned with the topic of quantum optimization, draws motivation from classical approaches such as BnB or the well-known construction of greedy solutions.

While classical solvers are well-established and powerful, the advent of quantum computing sparked interest in utilizing new computational primitives for optimization problems. Broadly speaking, there are several approaches to solving optimization problems using quantum computers. One such approach is quantum annealing, which uses the adiabatic theorem to ideally guarantee a quantum system to reach a desired ground state that encodes the solution to some combinatorial optimization problem [12]. Additionally, variational algorithms have been studied in great detail in recent years, such as Quantum Approximate Optimization Algorithm (QAOA) [13] or Variational Quantum Eigensolver (VQE) based solvers [14]. Both approaches seem promising on specific problems (which can be contrived), but exhibit challenges that need to be resolved before they can be useful [15]. Another group of quantum optimization routines is based on Amplitude Amplification (AA) [16, 17]. The idea in this context is to use the provable quadratic

[1] Here, simplicity refers to the single-constraint, single-objective structure of the knapsack problem, not its time-to-solution.

speedup of Grover's algorithm on unstructured problems [16] and utilize it to solve combinatorial optimization problems. In this thesis, AA based algorithms play the central role. In particular, Gilliam et al. [18] introduced Grover Adaptive Search (GAS), a quantum optimization algorithm that uses Grover's search to iteratively improve upon an incumbent solution.

GAS applies AA over the entire search space, and both its circuit depth and the number of required Grover iterations depend on the overlap between the prepared initial state and high-quality solutions. This motivates warm-starting, which initializes the quantum state in a biased superposition, often guided by classical heuristics, that concentrates amplitude on promising solutions and thereby reduces the amplification required to reach them [19]. For the knapsack problem, the Quantum Tree Generator (QTG), proposed by Wilkening et al. [20], realizes exactly such a state preparation. It is a quantum circuit that prepares a biased superposition of feasible solutions to the knapsack problem. In this thesis, the combination of GAS with the QTG as its warm-start forms the Baseline Approach (BA), to which we benchmark our contributions.

We design and study the Nested Amplitude Amplification (NAA), a nested approach to GAS warm-started by a QTG -inspired circuit. Nesting optimization algorithms have been proposed in the past [21, 22] and offer a rich landscape of design choices to be explored. The NAA first performs state preparation and AA on the first s of the knapsack's n items, biasing the quantum state toward promising partial assignments. The QTG then completes the remaining $n - s$ items, and the resulting state serves as the state preparation for a subsequent GAS over all n items. In this sense, the inner amplification stage hot-starts the outer GAS. The design of the state preparation operators, the criteria that mark promising partial states, exact gate counts and empirical benchmarking make up the main contributions of this thesis.

Concretely, this thesis aims to answer the question of whether the NAA can outperform the BA with respect to solution quality or gate count, and if so, in which regime. Answering this question gives insight into the gap between theoretical advantages gained by nesting quantum algorithms and practical performance improvements by nesting amplitude amplification, studied specifically on the knapsack problem.

Parts of this thesis have been published as a paper [23] at the IEEE Quantum Week 2026. It reports the results of Section 5.6, in a much simplified form of tracking gate costs.

Chapter 2 introduces the knapsack problem, Grover search, GAS, the QTG and quantum arithmetic primitives. The main theoretical contribution of this work can be found in Chapter 3, where the BA and the NAA are described in detail, including specific design choices of the quantum circuits, gate counts and amplification criteria. Chapter 4 serves to give a more in-depth description of implementation details (including classical amplitude tracking), the metrics employed to quantify performance and the instance generation. Finally, Chapter 5 presents the empirical results of the NAA and compares it to the BA, first in the biased setting that reproduces the parameter choices of [20] and then in an unbiased setting in which the NAA does show an advantage, before Chapter 6 concludes the thesis and outlines directions for future work.

2 Background

2.1 The Knapsack Problem

As outlined in Chapter 1 of this work, the knapsack problem is a modern staple in theoretical computer science, as well as a fundamental testbed in combinatorial optimization theory. The problem is considered important because of its widespread occurrence as a subproblem, e.g. in scheduling and resource allocation problems [24] as well as the fact that it is one of the simpler Nondeterministic Polynomial time (NP) problems [2]. "Simple" here does not refer to time-to-solution, but rather the single-constraint, single-objective nature of its formulation [11, 25].

Given a knapsack of capacity c and m items with weight w_m and a profit p_m, the knapsack problem consists of maximizing the total packed profit while respecting the capacity constraint:

$$\max_{x \in \{0,1\}^n} \quad \sum_{m=1}^{n} p_m x_m \tag{1}$$

$$\text{subject to} \quad \sum_{m=1}^{n} w_m x_m \leq c. \tag{2}$$

This formulation employs an encoding of the solution as a sequence of binary decision variables x_m, where $x_m = 1$ if item m is included in the knapsack and $x_m = 0$ otherwise. [25]

Classical approaches to solving knapsack instances include Dynamic Programming [11], Branch-and-Bound [6] and the Fully Polynomial-Time Approximation Scheme [10]. Previous work has established that the effective difficulty of solving knapsack instances heavily depends on the distribution of the weights and profits of the items[2]. Pisinger [9] showed that strong LP bounds are hard to find for instances with similar efficiencies $\frac{p_m}{w_m}$, making those instances harder to solve to optimality.

Given the fact that there are still classically hard regimes for the knapsack problem, it appears relevant for modern quantum algorithm research. The structure defined in (1)

[2]NP - hardness as a worst-case complexity statement guarantees that no polynomial-time algorithm solves all instances correctly, but makes no statement about the practical difficulty or runtime of typical instances, unless P=NP [26]

and (2) is well suited for the encoding to qubit registers [27]. Moreover, the simplicity of its formulation has sparked the knapsack problem to become a common benchmark for quantum algorithms, which in turn provides a larger body of literature to draw from [28, 29]. For these reasons, we consider the knapsack problem in this thesis, while maintaining generalizability of the presented approaches to other combinatorial optimization problems.

2.2 Grover's Algorithm

The field of quantum algorithms is not as recent a field as is often thought. After the discovery of Shor's algorithm (a quantum algorithm to factor integers into their prime factors) by Peter Shor in 1994 [30], much work has been done to find quantum algorithms that perform tasks with a better scaling behaviour than their best known classical counterparts. One such example with relevance to combinatorial optimization is Grover's algorithm, proposed by Lov Grover in 1996 [16]. Grover's algorithm is central to the field, with wide-ranging applications in optimization [17], cryptanalysis [31], and other areas [32], as well as a foundational influence on the design of quantum algorithms more broadly [33]. This section provides an introduction to the algorithm, whereas Section 2.4 introduces more recent advances of using the algorithm in the context of combinatorial optimization.

Suppose we are given a database of $N = 2^n$ items[3], and we are subsequently interested in the M solutions to the search problem, a set of items we call "marked" items. Furthermore, we are given access to an oracle function $f(x)$, which takes an item index as input:

$$f(x) = \begin{cases} 1 & \text{if item } \ x \ \text{ is marked} \\ 0 & \text{otherwise} \end{cases} \tag{3}$$

The search problem now consists of finding one of the marked items. [34]

Classically, brute-forcing the problem requires $O(N)$ queries to f. An intuitive way to understand this scaling is the fact that for $M = 1$, we must on average search $\frac{N}{2}$ items to find the solution. An important note to make in the classical setting is that the aforementioned scaling behaviour applies to unstructured search problems (i.e. no further

[3]This is to allow the indices to be stored in bitstrings of length n. In the general case, some indices mapped to bitstrings simply do not correspond to actual items in the database.

information about the problem is known). For structured searches, much more efficient classical algorithms have been proposed. [35]

Grover's algorithm now seeks to solve this search problem using quantum resources, encoding the item indices into an N-dimensional Hilbert space, in which each basis state corresponds to a unique item. To that end, suppose we are given access to a quantum implementation of f in the form of a unitary phase oracle O_f, that acts on the computational basis states as follows[4]:

$$O_f|x\rangle = (-1)^{f(x)}|x\rangle. \tag{4}$$

In this setting, O_f applies a distinct phase to all basis states that correspond to marked items. We call those states "marked states". Additionally, we can define a unitary operator D, that applies a phase to all states but the all-zero state $|0\rangle^{\otimes n}$:

$$D = 2(|0\rangle\langle 0|)^{\otimes n} - I. \tag{5}$$

We define the unitary operator $G = H^{\otimes n}DH^{\otimes n}O_f$ as the sequential application of O_f followed by D (modified by the Hadamard transform, this operator is referred to as the diffusion operator). A single application of G constitutes one "Grover iteration".

The algorithm operates on a search register of n qubits holding the item indices, prepared in the state $|0\rangle^{\otimes n}$, alongside the ancilla required by the oracle. The algorithm proceeds in the following steps:

1. Apply Hadamard gates to all n qubits of the search register, yielding the state $|s\rangle = H^{\otimes n}|0\rangle^{\otimes n}$, the equal superposition of all basis states.
2. Apply G repeatedly for $O\left(\sqrt{\frac{N}{M}}\right)$ iterations.
3. Measure the search register.

Following arguments made in detail in [34], the state after r iterations of G can be written as:

$$G^r|s\rangle = \sin((2r+1)\theta)|\alpha\rangle + \cos((2r+1)\theta)|\beta\rangle, \tag{6}$$

[4]The phase oracle is an abstraction. Implementing requires at least one ancilla, for instance initialised in $|-\rangle$ to obtain the phase by kickback. Since they are uncomputed, we leave them implicit here.

where $|\alpha\rangle$ is the equal superposition of the marked states, $|\beta\rangle$ is the equal superposition of the unmarked states and $\sin(\theta) = \sqrt{\frac{M}{N}}$. Following this expression, step 3 measures a marked state with probability of at least $1 - \frac{M}{N}$ after exactly $r = \left\lfloor \frac{\pi}{4\theta} \right\rfloor$ Grover iterations, showing the quadratic speedup compared to classical brute-force search [16]. As a matter of fact, the quadratic speedup is the optimal scaling behaviour on any unstructured search problem [36, 37].

2.3 Amplitude Amplification (AA)

Nowadays, Grover's algorithm is widely understood as a special case of a more broadly defined algorithm: Amplitude Amplification (AA). AA generalizes the idea of amplifying the amplitudes in a marked subspace and allows for a more general initial state. Particularly, we define a state preparation operator A that prepares some initial state, and an oracle operator O that marks the marked states with a phase. Amplitude amplification then applies the operator

$$\left(-ADA^\dagger O\right)^r A, \tag{7}$$

with r being the number of AA iterations and D as in (5). Measuring the system results in the measurement of a marked state with high probability after $O\left(\frac{1}{\sqrt{p}}\right)$ iterations, where p is the initial probability[5] of measuring a marked state in the state prepared by A [33, 38]. Grover's algorithm is recovered if $A = H^{\otimes n}$ and the oracle is constructed to mark all states that satisfy a boolean oracle f.

2.4 Grover Adaptive Search (GAS)

As shown in Section 2.3, AA speeds up the problem of database search quadratically over brute force search. In order to make AA applicable to an optimization setting, the algorithm needs to be adapted. Foundationally, Dürr and Høyer [17] proposed a quantum minimum-finding routine, which, generalized to arbitrary combinatorial optimization problems, is known as GAS [18]. GAS should be understood as a quantum-assisted sequential approximation method applied to a general optimization problem.

[5]Since the initial state need not be uniformly distributed, the optimal number of iterations does not simply depend on the number of marked states.

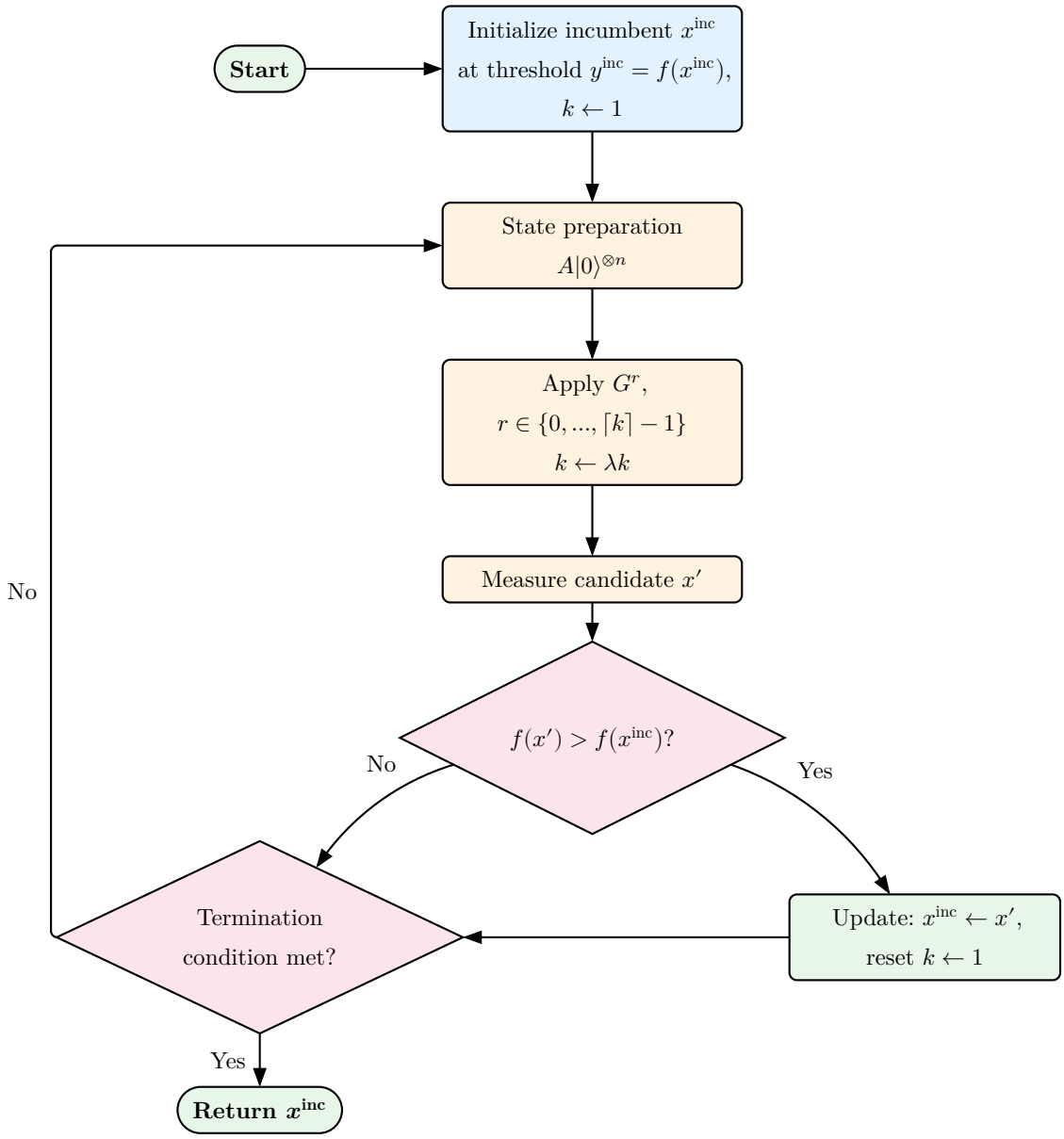


Figure 1: Flowchart of the Grover Adaptive Search (GAS) algorithm. After the initial state is prepared, the number of applications of G is sampled from an exponentially increasing interval. A successful measurement updates the incumbent. The procedure is repeated until some termination condition is met.

Fundamentally, GAS takes the following inputs: the cost function f of the problem, a state preparation operator A and an oracle $O(y^{\text{inc}})$, which may be dependent on the incumbent threshold y^{inc} and marks all states better than the incumbent w.r.t. f. As portrayed in Figure 1, the algorithm begins by identifying an initial incumbent x^{inc}, which is oftentimes the greedy solution to the problem. Following this step, the initial state

is prepared according to A. Importantly, should A prepare a superposition containing infeasible states, $O(y^{\text{inc}})$ must be constructed to mark all states that are both feasible and better than the incumbent. In this thesis, we focus on the preparation of a superposition of only feasible states. Due to the unknown probability mass of marked states at each update step of GAS, the ideal number of Grover iterations (as described in Section 2.2) is generally unknown. To overcome this limitation, the rotation count r is sampled uniformly from the set $\{0, 1, ..., \lceil k \rceil - 1\}$, whose upper bound is grown via $k \leftarrow \lambda k$ after each unsuccessful measurement. This approach maintains the quadratic advantage over brute-force search [16, 39]. While alternative sampling strategies may reduce the number of iterations, such improvements are bounded by constant factors [19, 40]. G is then applied r times and the resulting state is measured. The incumbent is updated if the measured solution is better than the incumbent w.r.t. f. The procedure is repeated until some termination condition is met, which may be a time limit, a maximum number of iterations or a convergence criterion. Further detail regarding the implementation of termination conditions in this thesis is provided in Section 3.6.

The pseudocode for GAS is reproduced in Algorithm 2, which calls the subroutine Algorithm 1 to perform the quantum search [18]. Notably, $A = H^{\otimes n}$ recovers a standard, iterative Grover Search (as described in Section 2.2).

Algorithm 1: QSearch

Require: State preparation A, oracle $O(y^{\text{inc}})$, incumbent x^{inc}, y^{inc}, k, λ
Randomly select the rotation count r from the set $\{0, 1, ..., \lceil k \rceil - 1\}$;
Apply Grover Search with r iterations, using A and $O(y^{\text{inc}})$. We denote the outputs $\tilde{x}$ and $\tilde{y}$ respectively;
if $\tilde{y} > y^{\text{inc}}$ **then**
    $x^{\text{inc}} = \tilde{x}$, $y^{\text{inc}} = \tilde{y}$, and $k = 1$
end
else
    $x^{\text{inc}} = x^{\text{inc}}$, $y^{\text{inc}} = y^{\text{inc}}$, and $k = \lambda k$
end
return x^{inc}, y^{inc}, k, r

Algorithm 2: Grover Adaptive Search

Require: $f : X \to \mathbb{R}$, $\lambda > 1$
Uniformly sample $x^{\text{inc}} \in X$ and set $y^{\text{inc}} = f(x^{\text{inc}})$;
Set $k = 1$ and $i = 1$;
repeat
    $x^{\text{inc}}, y^{\text{inc}}, k, r \leftarrow \text{QSearch}(A, O(y^{\text{inc}}), x^{\text{inc}}, y^{\text{inc}}, k, \lambda)$
    $i = i + 1$;
until a termination condition is met;

2.5 Quantum Arithmetic Circuits

Many quantum algorithms require arithmetic operations on values stored in superposition. This section introduces some fundamental building blocks for quantum arithmetic: addition in Fourier space and integer comparison. These concepts will be essential for the QTG introduced in Section 2.6, as a means to specify a feasible state preparation operator.

Throughout this work, we adopt the notion of gate count from Wilkening et al., where "all single-qubit gates, single-controlled single-qubit rotations, and Toffoli gates together constitute a universal set of elementary gates of equal implementation cost" [20]. The gate count is then defined as the number of such elementary gates in a quantum circuit. This definition does not address issues that may arise from gate decompositions into native gatesets or the difference in difficulty of implementing single and two qubit gates. However, it is a useful metric to compare the complexity of quantum circuits in a hardware agnostic way.

2.5.1 Quantum Fourier Transform (QFT) Adders

A fundamental operation in quantum arithmetic is adding a classical number v to a quantum register encoding some number a in its binary representation $|a\rangle = \left|\tilde{a}_{|a|}\tilde{a}_{|a|-1}...\tilde{a}_1\right\rangle$, where $|a|$ denotes the number of qubits in the register. This reduces to obtaining the state $\left|(a+v) \bmod 2^{|a|}\right\rangle$.

To achieve this, we utilize a Draper adder, first introduced by Draper in [41]. The adder first applies a Quantum Fourier Transform (QFT) [34, 42] to all qubits of $|a\rangle$, yielding:

$$\text{QFT } |a\rangle = \frac{1}{\sqrt{2^{|a|}}} \sum_{k=0}^{2^{|a|}-1} e^{\frac{2\pi i a k}{2^{|a|}}} |k\rangle = \frac{1}{\sqrt{2^{|a|}}} \bigotimes_{k=1}^{|a|} \left(|0\rangle + e^{\frac{2\pi i a}{2^k}} |1\rangle\right). \tag{8}$$

The value of a is now encoded in the phases of the qubits. A key insight is, that in Fourier space, additions are multiplicative:

$$e^{\frac{2\pi i a}{2^k}} e^{\frac{2\pi i v}{2^k}} = e^{\frac{2\pi i (a+v)}{2^k}}. \tag{9}$$

Decomposing the classical number v into its binary representation $v = \sum_{j=1}^{|a|} \tilde{v}_j 2^{j-1}$, we can apply a series of phase gates to the state $|a\rangle$ in Fourier space. Following (9), we need to multiply the k-th qubit's phase in $|a\rangle$ by the term

$$e^{\frac{2\pi i v}{2^k}} = e^{\frac{2\pi i \sum_{j=1}^{|a|} \tilde{v}_j 2^{j-1}}{2^k}} = \prod_{j=1}^{|a|} e^{\frac{2\pi i \tilde{v}_j 2^{j-1}}{2^k}} = \prod_{j=1}^{k} e^{\frac{2\pi i \tilde{v}_j}{2^{k-j+1}}}. \tag{10}$$

In order to multiply the term specified in (10) to the $|1\rangle$ part of the k-th qubit, we define the unitary operator $R_m(b)$ for a bit $b \in \{0, 1\}$ as follows:

$$R_m(b) = \mathrm{diag}\Big(1, e^{\frac{2\pi i b}{2^m}}\Big). \tag{11}$$

Recognizing that only terms where $j \leq k$ contribute to the phase and the fact that all $R_m(b)$ are diagonal matrices, we can apply the following product of rotation gates to act on the k-th qubit of the state $|a\rangle$ in Fourier space:

$$R_k(v) = \prod_{j=1}^{k} R_{k-j+1}(\tilde{v}_j) = \mathrm{diag}\left(1, e^{2\pi i \sum_{j=1}^{k} \frac{\tilde{v}_j}{2^{k-j+1}}}\right) = \mathrm{diag}\left(1, e^{2\pi i \frac{v \bmod 2^k}{2^k}}\right). \tag{12}$$

This gate equals the identity I whenever $\frac{v \bmod 2^k}{2^k} \in \mathbb{Z}$, which occurs exactly when $v \bmod 2^k = 0$, i.e., when the lowest k bits of v are all zero. Denoting by $\mathrm{LSO}(v)$ the 1-based position of the least significant one in the binary representation of v, $R_k(v)$ is the identity operation for all qubits $k < \mathrm{LSO}(v)$. Hence $R_k(v)$ is non-trivial $\max(0, |a| - \mathrm{LSO}(v) + 1)$ times.

Applying the rotations defined in (12) yields:

$$\frac{1}{\sqrt{2}^{|a|}} \bigotimes_{k=1}^{|a|} \Big(|0\rangle + e^{\frac{2\pi i (a+v)}{2^k}} |1\rangle\Big). \tag{13}$$

The state $\big|(a+v) \bmod 2^{|a|}\big\rangle$ can be recovered by applying an inverse QFT. From this derivation, we can derive the gate count of adding a classical number v to a quantum register storing a:

$$G^{|a|}_{\mathrm{ADD}_v} = 2G^{|a|}_{\mathrm{QFT}} + \max(0, |a| - \mathrm{LSO}(v) + 1), \tag{14}$$

where $G^{|a|}_{\mathrm{QFT}} = |a|\frac{|a|+1}{2}$ is the gate count for the QFT on $|a|$ qubits [20].

For reasons that are further explained in Section 3.5.2, it is also of importance to understand how the Draper adder generalises to adding a classical number v multiplied

by a quantum register $|q\rangle$ to a target register $|a\rangle$. In essence, we want to obtain the state $\left|(a + v \cdot q) \bmod 2^{|a|}\right\rangle$. $|q|$ and $|a|$ denote the number of qubits in the respective registers.

This is a direct generalisation from the argument above, as we can again utilise binary representation to write:

$$v \cdot q = \sum_{i=1}^{|q|} \tilde{q}_i \cdot v \cdot 2^{i-1}. \tag{15}$$

By requiring the i-th qubit of $|q\rangle$ to be in the state $|1\rangle$, we recover the case from (12). Hence, the addition can be implemented by applying a controlled rotation gate $CR_k(v \cdot 2^{i-1})$ to the k-th qubit of $|a\rangle$, controlled on the i-th qubit of $|q\rangle$. The gate count for this operation is given by:

$$\begin{aligned} G^{|a|}_{\mathrm{ADD}_{vq}} &= 2G^{|a|}_{\mathrm{QFT}} + \sum_{i=1}^{|q|} \max(0, |a| - \mathrm{LSO}(v \cdot 2^{i-1}) + 1) \\ &= 2G^{|a|}_{\mathrm{QFT}} + \sum_{i=1}^{|q|} \max(0, |a| - \mathrm{LSO}(v) - i + 2), \end{aligned} \tag{16}$$

using the fact that $\mathrm{LSO}(v \cdot 2^{i-1}) = \mathrm{LSO}(v) + i - 1$.

2.5.2 Integer Comparison

Another arithmetic primitive of importance especially to Section 2.6 is the ability to compare the value a stored in a quantum register $|a\rangle$ to a classical number v and control the flow of a quantum circuit based on the result of this comparison. Two comparisons are of essence:

1. **Threshold comparison:** determining whether $a \geq v$ (or $a > v$)
2. **Zero test:** checking whether $|a\rangle = |0\rangle^{\otimes n}$

We begin with the strict threshold comparison and describe the subcircuit that implements some unitary U if $a > v$, where a is a value stored in a quantum register and v is a classical threshold. Let $|a|$ denote the number of qubits in the register storing a, and let $\ell(v) = \lfloor \log_2 v \rfloor + 1$ denote the bit-length of the classical value v. We write $\left(\tilde{v}_{\ell(v)}, \tilde{v}_{\ell(v)-1}, ..., \tilde{v}_1\right)$ for the binary representation of v, and $\left(\tilde{a}_{|a|}, \tilde{a}_{|a|-1}, ..., \tilde{a}_1\right)$ for the binary representation of the value in the register. The quantum circuit implementing the

comparison must find the most-significant bit j, where the binary representations of a and v differ, i.e., the largest j such that $\tilde{a}_j \neq \tilde{v}_j$. Three such cases can occur:

1. $\tilde{a}_j = 1$ and $\tilde{v}_j = 0$: In this case, $a > v$ and the unitary U should be applied,
2. $\tilde{a}_j = 0$ and $\tilde{v}_j = 1$: In this case, $a < v$ and the unitary U should not be applied,
3. If no such j exists, $a = v$ and the unitary U should not be applied.

For the QTG and the GAS oracle sketched in Section 2.4, it is sufficient to consider only the first case. The comparison $a > v$ holds exactly if there exists a bit position m where $\tilde{a}_m = 1$, $\tilde{v}_m = 0$, and all higher bits match. Since v is classical, we enumerate all positions m where $\tilde{v}_m = 0$ and control the application of U on the m-th qubit being $|1\rangle$ while requiring all higher qubits $i > m$ to match $\tilde{v}_i$. Since only one bit can be the most significant differing bit, this case is well defined. Wilkening et al. [20] formalise this insight using orthogonal disjunctive normal forms, providing a gate count for the comparison:

$$G_{>v,1}^{|a|} = \sum_{i=1}^{\ell(v)} (1 - \tilde{v}_i) \cdot (2(|a| - i) + 1), \tag{17}$$

$$G_{>v,2}^{|a|} = 1 + \sum_{i=1}^{\ell(v)} (\widetilde{v+1})_i \cdot (2(|a| - i) + 1), \tag{18}$$

$$G_{>v}^{|a|} = \min\Big(G_{>v,1}^{|a|}, G_{>v,2}^{|a|}\Big). \tag{19}$$

$G_{>v,1}^{|a|}$ indicates the gate count using the strategy described above, whereas $G_{>v,2}^{|a|}$ formalises the gate count for the opposing case, where U is applied unconditionally and its inverse is then applied if $a \leq v$. As the exact bitstrings dictate which gate count is lower, we take the minimum over both strategies. [20, 43]

The comparison $a \geq v$ on integer values is equivalent to $a > v - 1$. Following this logic, the gate counts of (17) to (19) only need to be slightly modified:

$$G_{\geq v,1}^{|a|} = \sum_{i=1}^{\ell(v)} \Big(1 - (\widetilde{v-1})_i\Big) \cdot (2(|a| - i) + 1), \tag{20}$$

$$G_{\geq v,2}^{|a|} = 1 + \sum_{i=1}^{\ell(v)} \tilde{v}_i \cdot (2(|a| - i) + 1), \tag{21}$$

$$G_{\geq v}^{|a|} = \min\Big(G_{\geq v,1}^{|a|}, G_{\geq v,2}^{|a|}\Big). \tag{22}$$

As discussed briefly above, the minimum across two different strategies (apply U if $a \geq v$ or apply U unconditionally and then apply its inverse if $a < v$) is taken to determine the gate count of the comparison.

Finally, we turn to the zero test. In order to be able to apply Grover Search as discussed in Section 2.2, we need to be able to apply the unitary defined in (5). Generally, this requires the ability to execute some unitary U (a phaseflip gate in this case) if the value of the target register is equal to zero. One multi-controlled operation $C^{|a|}U$ can be used to implement this, where $|a|$ is the number of qubits in the register storing a. Using the decomposition described in Section 2.5, this $|a|$-controlled gate yields a gate count of:

$$G_{=0}^{|a|} = 2|a| - 1, \tag{23}$$

requiring $|a| - 1$ ancilla qubits [20].

2.6 Quantum Tree Generator (QTG)

Feasible state preparation routines have grown in importance in the context of quantum optimization algorithms, both to boost performance of AA based protocols, but also variational algorithms like QAOA. [19, 20, 44, 45] Section 2.4 displays the need for such a state preparation operator A, especially in the context of knapsack problems.

For the knapsack problem, a feasible state preparation routine called QTG has been introduced by Wilkening et al. [20]. The QTG is a quantum circuit that generates a (potentially biased) superposition of all feasible states for a knapsack problem as defined in Section 2.1. The following sections introduce the QTG circuit structure, its interpretation as a classical state sieve, and the resulting gate counts based on the primitives introduced in Section 2.5.

2.6.1 QTG as a Quantum Circuit

This subsection serves to explain in detail how the QTG works and is implemented as a quantum circuit for a knapsack problem as detailed in Section 2.1. The QTG begins by initializing three registers. The first register $|q\rangle$ represents one decision variable x_m for each item in the knapsack problem, n in total. Furthermore, a remaining capacity register

$|C\rangle$ is initialized to the capacity of the knapsack c, using $|C| = \ell(c) = \lfloor \log_2(c) \rfloor + 1$ qubits. The third register is a profit register $|P\rangle$ initialized to 0, which stores the total profit of the items added to the knapsack, using $|P| = \ell\left(\sum_{m=1}^{n} p_m\right) = \left\lfloor \log_2\left(\sum_{m=1}^{n} p_m\right) \right\rfloor + 1$ qubits.

For each item m, the QTG first applies a biased Hadamard gate H_b to the qubit $|q_m\rangle$ containing decision variable x_m, controlled on whether the remaining capacity register C is greater than or equal to the weight of the item w_m. This can be understood as the check of whether or not item m can be added to the solution without violating the capacity constraint. If the item can be added, the amplitude of the state is divided between the two branches of the decision variable x_m, according to the bias parameter b. In particular, the QTG uses a reference solution x^{ref}, towards which it biases the amplitude distribution. Let x_m^{ref} denote the m-th bit of the reference solution. The biased Hadamard gate $H_b^{x_m^{\text{ref}}}$ is then defined as:

$$H_b^0 = R_y\left(2 \arccos \sqrt{\frac{1+b}{2+b}}\right) = \frac{1}{\sqrt{2+b}} \begin{pmatrix} \sqrt{1+b} & -1 \\ 1 & \sqrt{1+b} \end{pmatrix} \tag{24}$$

$$H_b^1 = R_y\left(2 \arccos \sqrt{\frac{1}{2+b}}\right) = \frac{1}{\sqrt{2+b}} \begin{pmatrix} 1 & -\sqrt{1+b} \\ \sqrt{1+b} & 1 \end{pmatrix} \tag{25}$$

If the reference solution excludes item m ($x_m^{\text{ref}} = 0$), the gate H_b^0 biases the amplitude towards $|0\rangle$ (not including the item). Conversely, if the reference solution includes item m ($x_m^{\text{ref}} = 1$), the gate H_b^1 biases towards $|1\rangle$ (including the item). For $b = 0$, both variants reduce to the standard Hadamard gate [20]. If item m does not fit ($w_m > C$), no superposition is created and the state remains unchanged with the item excluded.

After having applied the biased Hadamard gate, the capacity register C is updated by subtracting the weight of the item w_m, controlled on the decision variable x_m being one and hence the item being included. Lastly, the profit register P is updated by adding the profit of the item p_m, again controlled on the decision variable x_m being one.

A single step of the QTG processing item m can be thought of as an operation creating a superposition of including and excluding an item from the solution, insofar as it is feasible. Given a partial state $|x_1 ... x_{m-1} 0 ... 0\rangle |C\rangle |P\rangle$ where C is the remaining capacity and P the accumulated profit after $m-1$ items, the effect of the m-th layer is:

$$|x_1...x_{m-1}0...0\rangle|C\rangle|P\rangle \longrightarrow \sqrt{\mathcal{P}}|x_1...x_{m-1}0...0\rangle|C\rangle|P\rangle$$
$$+\sqrt{1-\mathcal{P}}|x_1...x_{m-1}10...0\rangle|C-w_m\rangle|P+p_m\rangle, \tag{26}$$

where the probability $\mathcal{P}$ depends on the bias b, the reference solution x^{ref}, and whether item m fits into the remaining capacity. Specifically:

$$\mathcal{P} = \begin{cases} \frac{1+b}{2+b} \text{ if } w_m \leq C \text{ and } x_m^{\text{ref}} = 0 \\ \frac{1}{2+b} \text{ if } w_m \leq C \text{ and } x_m^{\text{ref}} = 1 \\ 1 \quad \text{if } w_m > C. \end{cases} \tag{27}$$

Putting it all together, the QTG iterates through all n items, resulting in a superposition of all feasible states, weighted by the bias parameter b. The final state of the QTG is given by:

$$\sum_{\substack{x\in\{0,1\}^n \\ w^Tx\leq c}} \alpha_x|x\rangle\big|c-w^Tx\big\rangle\big|p^Tx\big\rangle, \tag{28}$$

where the bias parameter b, reference solution x^{ref} and the structure of infeasible solutions in the solution space determine the amplitudes α_x of the feasible states. [20, 23, 44]

The pseudocode for the QTG is given in Algorithm 3. The shorthand $C^{\text{reg}}_{\text{cond}}(U)$ denotes a gate U applied to a register, controlled on condition *cond* evaluated on register *reg*.

Algorithm 3: Quantum Tree Generator

Ensure: Quantum state encoding feasible solution space
Initialize three quantum registers:
  Decision register $|q\rangle$: $|0\rangle^{\otimes n}$
  Capacity register $|C\rangle$: $|c\rangle$ ▷ Remaining capacity
  Profit register P: $|0\rangle$ ▷ Total profit
for $m = 1, ..., n$ **do**
  $C^C_{\geq w_m}\left(H_b^{x_m^{\text{ref}}}\right)$ ▷ Biased Hadamard on q_m, controlled on $w_m \leq C$
  $C^C_{x_m}\left(\text{SUB}_{w_m}\right)$ ▷ $C \leftarrow C - w_m$, controlled on q_m
  $C^P_{x_m}\left(\text{ADD}_{p_m}\right)$ ▷ $P \leftarrow P + p_m$, controlled on q_m
end for
return $\sum_{\substack{x\in\{0,1\}^n \\ w^Tx\leq c}} \alpha_x|x\rangle\big|c-w^Tx\big\rangle\big|p^Tx\big\rangle$

Using the explanations and results from Section 2.5, the QTG can be implemented as a quantum circuit as shown in Figure 2. While implementation details like the controlled gate on a comparison for the Hadamard gates or the Adder is abstracted for simplicity, the details can be found in Section 2.5.1 and Section 2.5.2.

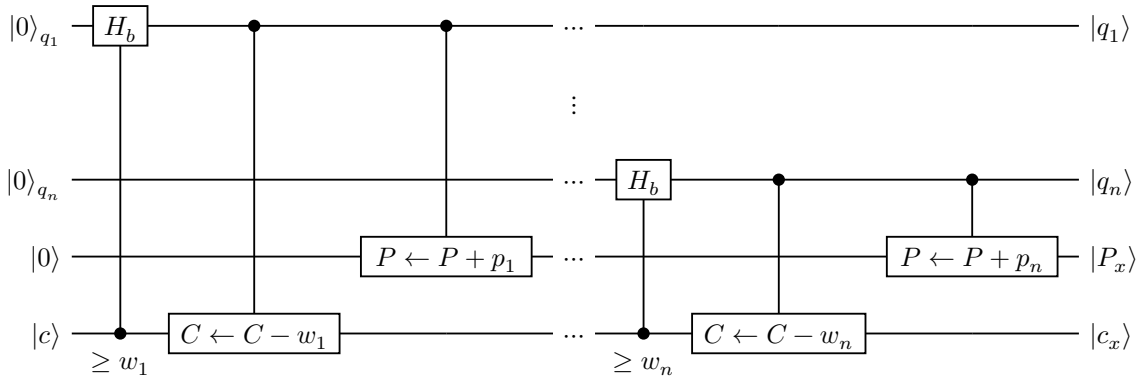


Figure 2: Quantum circuit of the Quantum Tree Generator (QTG) for n decision variables. For each decision variable, three steps are performed: First, controlled on whether the remaining capacity register C is greater than or equal to the weight of the item being added (implemented via the comparator circuit from Section 2.5.2), a (biased) Hadamard gate is applied to divide the amplitude between the two branches of the decision variable. Second, controlled on the decision variable being set to 1 (i.e. the item being added to the solution), the weight of the item is subtracted from the remaining capacity register C using QFT-based arithmetic (Section 2.5.1). Third, controlled on the decision variable being set to 1, the profit of the item is added to the profit register P. The result is a superposition of all feasible states.

2.6.2 QTG as a Classical State Sieve

As noted by Wilkening et al. [20], the QTG can be dequantized[6] to a quantum-inspired Classical Tree Generator (CTG). The CTG defines a classical routine to sample feasible bitstrings at the same distribution as the QTG. The pseudocode can be found in Algorithm 4.

We should note here, that the recursive structure of the CTG portrays the fact, that even if the algorithm operates at no bias, $b = 0$, the resulting probability distribution of feasible states is not uniform. This is due to the fact, that the CTG (and hence the QTG) samples feasible states in a depth-first manner, and hence the probability of a feasible state is proportional to the number of infeasible states that are pruned by the algorithm

[6]Or rather, the QTG is the quantized version of a classical routine.

on the same subtree. This dequantizability motivates a classical benchmarking routine in Section 4.2.

Algorithm 4: Classical Tree Generator (CTG)

Require: Reference solution x^{ref}, bias $b \geq 0$; entry call CTG$(1, c)$
function CTG(m, C):
  if $m > n$ **then**
    return $(x_1, ..., x_n)$
  end if
  if $w_m \leq C$ **then**
    if $x_m^{\text{ref}} = 1$ **then** set $x_m = 1$ with probability $\frac{1+b}{2+b}$
    else set $x_m = 1$ with probability $\frac{1}{2+b}$
    end if
  else
    $x_m = 0$
  end if
  return CTG$(m + 1, C - w_m x_m)$

2.6.3 Gate Count for the QTG

Being able to benchmark more elaborate approaches to QTG based GAS requires a detailed understanding of the gate count for the QTG with respect to certain parameters of an input problem. This section presents an overview over the gate count derived in [20], and adopts some of the conventions presented there, mostly to ensure comparability of results later on.

As discussed in Section 2.6.1, the QTG consists of one comparator subcircuit, one Adder subcircuit and one Subtract subcircuit for each of the n decision variables. The comparator for an item m contributes gates as described in (20) to (22):

$$G^{|C|}_{\geq w_m,1} = \sum_{i=1}^{\ell(w_m)} \left(1 - (\widetilde{w_m - 1})_i\right) \cdot (2(|C| - i) + 1), \tag{29}$$

$$G^{|C|}_{\geq w_m,2} = 1 + \sum_{i=1}^{\ell(w_m)} \widetilde{w_m}_i \cdot (2(|C| - i) + 1), \tag{30}$$

$$G^{|C|}_{\geq w_m} = \min\left(G^{|C|}_{\geq w_m,1}, G^{|C|}_{\geq w_m,2}\right). \tag{31}$$

The Adder on the profit register contributes $2G^{|P|}_{\text{QFT}} = |P|(|P| + 1)$ gates to perform the QFT, since the profit register can stay in Fourier space throughout the state preparation. However, since the comparator needs access to the capacity register $|C\rangle$ in the computa-

tional basis, the Subtractor for the capacity cannot leave $|C\rangle$ in Fourier space, and hence contributes $2(n-1)G_{\mathrm{QFT}}^{|C|} = (n-1)|C|(|C|+1)$ gates[7].

We are now left having to consider the contributions of the rotational gates from (12) by both the addition and the subtraction subcircuits. Section 2.5.1 shows that adding a classical value v to a register of size $|a|$ requires $|a| - \mathrm{LSO}(v) + 1$ non-trivial rotation gates, where $\mathrm{LSO}(v)$ denotes the position of the least significant one in the binary representation of v. However, each non-trivial rotation from (12) for either the addition or the subtraction is controlled on the same qubit q_m. Wilkening et al. [20] note that in order to optimize for parallel execution, the copying of the control qubits can be of help. We adapt this idea.

For t rotation gates controlled on the same qubit, this parallelization requires:

1. **Copy**: $(t-1)$ Controlled NOT (CNOT) gates to copy the control qubit to ancilla qubits,
2. **Rotations**: t rotation gates,
3. **Uncopy**: $(t-1)$ CNOT gates to uncompute step 1.

This yields a total of

$$3t - 2 = 3(|a| - \mathrm{LSO}(v)) + 1 \tag{32}$$

gates for a single addition of the value v, using (32).

For one item m however, the QTG applies both a profit addition and a capacity subtraction, and both are controlled on the same qubit q_m, encoding the decision variable x_m. Instead of copying q_m twice to ancillas, we copy it once, execute all rotations for both the addition and the subtraction, and uncompute once. Following (32), the profit addition uses $|P| - \mathrm{LSO}(p_m) + 1$ non-trivial rotation gates and the capacity subtraction uses $|C| - \mathrm{LSO}(w_m) + 1$ non-trivial rotation gates.

The gate count for item $m < n$ then has two contributions. The rotations themselves amount to a gate count of

$$|P| - \mathrm{LSO}(p_m) + |C| - \mathrm{LSO}(w_m) + 2 \tag{33}$$

[7]The last subtractor does not need to be applied as the last capacity constraint for item n has already been checked.

gates. Additionally, copying the control qubit $|q_m\rangle$ and later uncomputing it adds

$$2(\max(|P|, |C|) - \min(\mathrm{LSO}(p_m), \mathrm{LSO}(w_m))) \tag{34}$$

gates, since this operation is dominated by whichever register is larger. For the final item n, we do not need to perform a capacity update, so only its profit addition remains. The rotations contribute $|P| - \mathrm{LSO}(p_n) + 1$ gates and its copy and uncopy add $2(|P| - \mathrm{LSO}(p_n))$ gates.

Combining all components, the total gate count for the QTG on n variables is:

$$\begin{aligned} G^n_{\mathrm{QTG}} = &\sum_{m=1}^{n} G^{|C|}_{\geq w_m} + 2G^{|P|}_{\mathrm{QFT}} + 2(n-1)G^{|C|}_{\mathrm{QFT}} \\ &+2\sum_{m=1}^{n-1}(\max(|P|, |C|) - \min(\mathrm{LSO}(p_m), \mathrm{LSO}(w_m))) \\ &+\sum_{m=1}^{n-1}(|P| - \mathrm{LSO}(p_m) + |C| - \mathrm{LSO}(w_m) + 2) \\ &+2(|P| - \mathrm{LSO}(p_n)) + |P| - \mathrm{LSO}(p_n) + 1. \end{aligned} \tag{35}$$

2.6.4 A note on bias

Choosing a bias value b for the QTG is a non-trivial task. As discussed in Section 2.6.1, a bias of 0 recovers standard Hadamard gates, which do not necessarily produce a uniform superposition as shown in Section 2.6.2. Given some optimal solution x^{opt} and some reference solution x^{ref}, the bias plays an important role in how well the QTG works as a state preparation routine for GAS. Let

$$\Delta := \Delta\big(x^{\mathrm{opt}}, x^{\mathrm{ref}}\big) = \sum_{m=1}^{n} \big|x^{\mathrm{opt}}_m - x^{\mathrm{ref}}_m\big| \tag{36}$$

denote the Hamming distance between the two, i.e. the number of decision variables on which the reference solution disagrees with the optimum. Since the QTG biases each of the n biased Hadamard gates towards x^{ref}, exactly those Δ items whose reference bit is wrong receive the disfavoured amplitude $\sqrt{1/(2+b)}$, so Δ controls how much amplitude the optimal solution retains. In particular, Wilkening et al. [20] show that the bias can be chosen such that the amplitude of the optimal solution x^{opt} is maximized, by using:

$$b_{\text{opt}} = \frac{n}{\Delta} - 2 \approx \frac{n}{\Delta}. \tag{37}$$

This result is derived in [20] by establishing a lower bound on the sampling probability $|\alpha_x^{\text{opt}}|^2 = \left(\frac{b+1}{b+2}\right)^{n-\Delta} \cdot \left(\frac{1}{b+2}\right)^{\Delta}$, which is tight when all possible branchings occur. The optimal bias then follows from differentiating this expression with respect to b. In Section 5.1 this thesis examines average values for Δ to determine empirical values for b_{opt} and compare them to results in [20].

3 Nested Amplitude Amplification (NAA) Algorithm Design

3.1 Motivation

With Section 2.2 thoroughly covering Grover Search, we now motivate the main contribution of the thesis: a nested approach to GAS for the binary knapsack problem. Nested approaches more broadly have been proposed in literature, both in the context of specific problems, such as tree search [21], and in the context of general search and optimization algorithms [22, 46]. The motivation behind constructing a nested approach to Grover Search stems from the fact that, for particularly structured problems, one can hope to extract a large and helpful amount of information towards the solution of the problem, while only considering a fraction of the solution space. We could view this hope in the light of the most extreme case, where, for example, a binary knapsack problem of size n can be fully solved by considering $s < n$ items (e.g. because the first s items, sorted by value-to-weight ratio, already constitute or closely approximate the optimal solution). In detail, the hope is that partial amplification on the first s decision variables can bias the quantum state toward promising regions of the search space before the global search begins. The partial amplification increases the amplitude of states where the first s items are assigned favorably, so that when the subsequent global Grover search is applied, it operates on an advantageous superposition. This motivates the idea outlined in the following sections.

3.2 Baseline Approach (BA)

The BA [20] serves as the benchmark for the algorithm developed in this thesis. It is a combination of concepts from QTG and GAS. It operates on a knapsack instance as described in Section 2.1 and a provided incumbent solution x^{inc}, with incumbent profit $y^{\text{inc}} = \sum_{m=1}^{n} p_m x_m^{\text{inc}}$.

3.2.1 Implementation

In essence, the BA applies GAS to the knapsack problem, with a state preparation operator $A_n := \text{QTG}_n$ that acts on the whole search space of n decision variables. In particular, following Section 2.4, the BA applies the operator

$$\left(-A_n D_n A_n^{\dagger} O_n(y^{\text{inc}})\right)^r A_n \tag{38}$$

for each of the AA steps, where $O_n(y^{\text{inc}})$ is the oracle on the full state space, D_n the operator on that space as described in (5), and r the number of Grover iterations. An AA step is defined in the light of Figure 1, such that each possible update of the incumbent x^{inc} is considered an AA step (the procedure is still called an AA step, even if the measured solution is not better than the incumbent). The quantum circuit for one AA step can be seen in Figure 3, operating on the decision register $|q\rangle$ of n qubits initialized to $|0\rangle^{\otimes n}$, the remaining capacity register $|C\rangle$ of $|C|$ qubits initialized to $|c\rangle$, and the profit register $|P\rangle$ of $|P|$ qubits initialized to $|0\rangle$, as defined in Section 2.6.1.

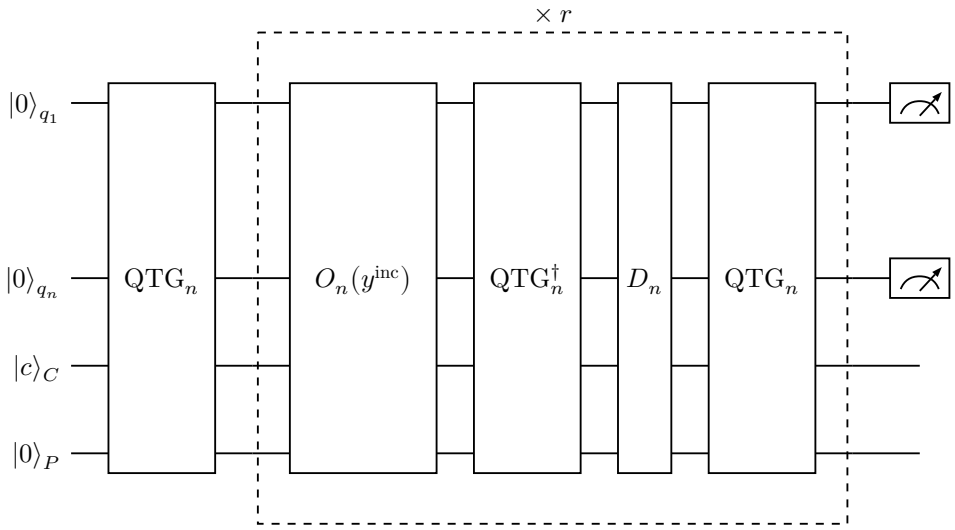


Figure 3: Quantum circuit of the Baseline Approach (BA) for one Amplitude Amplification (AA) step of Grover Adaptive Search (GAS). The state preparation $A_n \coloneqq \text{QTG}_n$ initializes a superposition over all feasible solutions. The dashed box indicates the Grover iteration repeated r times: oracle $O_n(y^{\text{inc}})$ marks states with profit exceeding y^{inc}, followed by the diffusion operator $A_n D_n A_n^\dagger$.

The oracle $O_n(y^{\text{inc}})$ marks all states with a profit exceeding that of the incumbent, y^{inc}. In detail,

$$O_n(y^{\text{inc}})|x\rangle = \begin{cases} -|x\rangle & \text{if} \quad \sum_{m=1}^{n} p_m x_m > y^{\text{inc}} \\ |x\rangle & \text{otherwise} \end{cases} \tag{39}$$

The implementation of this oracle can be fully realised using the comparator circuit from Section 2.5.2. The gate count of this oracle is hence

$$G^{|P|}_{>y^{\text{inc}}} = \min\Big(G^{|P|}_{>y^{\text{inc}},1}, G^{|P|}_{>y^{\text{inc}},2}\Big), \tag{40}$$

with definitions of $G^{|P|}_{>y^{\text{inc}},1}$ and $G^{|P|}_{>y^{\text{inc}},2}$ as in (17) and (18).

3.2.2 Gate Count

In order to benchmark against the BA, we are interested in the gate count of the approach from Section 3.2.1, per AA step. Throughout this thesis, we use G to denote per-step or per-shot gate counts and $\mathcal{C}$ for accumulated costs over multiple steps. The gate count is a function of the knapsack problem (with parameters n, c and all profits $\{p_m\}, \forall m \in \{1, ..., n\}$), the incumbent solution x^{inc} (with profit y^{inc}) and the number of Grover iterations r, sampled for the AA step in question. Following the circuit in Figure 3, the gate count of the BA for a single AA step with r Grover iterations is given by:

$$G_{\text{baseline}}(r) = (2r+1)G^n_{\text{QTG}} + r\Big(G^n_{=0} + G^{|P|}_{>y^{\text{inc}}}\Big), \tag{41}$$

with G^n_{QTG} from (35), $G^n_{=0}$ from (23), and $G^{|P|}_{>y^{\text{inc}}}$ from (19). The state preparation $A_n = \text{QTG}_n$ is applied $2r+1$ times, since each of the r Grover iterations invokes it twice, the state needs to be prepared once and each iteration additionally contributes one diffusion $G^n_{=0}$ and one oracle $G^{|P|}_{>y^{\text{inc}}}$.

A full run of the BA consists of a sequence of AA steps, terminated by a termination criterion t_{GAS} as described in Section 3.6.2. Let N_{steps} denote the (random) number of AA steps performed until termination, and let r_t be the number of Grover iterations sampled by GAS at step t. The total gate cost of the BA is the sum of the per-step costs over all sampled iteration counts,

$$\mathcal{C}_{\text{baseline}} = \sum_{t=1}^{N_{\text{steps}}} G_{\text{baseline}}(r_t), \tag{42}$$

with $G_{\text{baseline}}(r_t)$ as in (41).

3.3 General Nested Amplitude Amplification

Now that the BA approach is introduced, we turn our attention towards the general NAA approach to the binary knapsack problem, as motivated in Section 3.1. Generally, the approach is the following: instead of performing the BA on the whole search space of n variables, we initially perform GAS on the first $s < n$ decision variables, called inner stage, amplifying marked partial states (which are defined in detail in Section 3.5). After

the partial amplification, we then perform a GAS on the whole search space, called outer stage. Because marked states on the entire search space have already been partially amplified, the outer stage starts from a potentially favourable superposition and could reach a marked state with fewer Grover iterations than the BA. In a nutshell, we could identify the procedure as a GAS, hot-started by another AA based algorithm.

The NAA procedure given in Algorithm 6, decomposes into two routines. The Inner Iteration Finder (IIF), given in Algorithm 5, performs a partial state preparation followed by partial amplification on the first s decision variables, and returns a suitable number of inner Grover iterations r_{in} together with the cost $\mathcal{C}_{\text{IIF}}$ of settling on that r_{in}. The outer stage completes the QTG on the remaining $n - s$ items and uses the inner stage as a state preparation for a GAS on all n items. The oracle used by the outer stage is the same $O_n(y^{\text{inc}})$ already introduced in (39) for the BA.

We adopt the following conventions throughout this thesis: the IIF operates on the partial state space, spanned by the first $s < n$ decision variables, sometimes referred to as partial depth s[8]. Partial states are bitstrings of length s, denoted $x_{1:s}$. On this partial state space, the partial state preparation A_s^q prepares a superposition over feasible partial assignments, and the partial oracle $O_s^q(y^{\text{inc}})$ marks promising partial states. Both A_s^q and O_s^q depend on the criterion q for marked partial states and are defined concretely in Section 3.5. The remaining $n - s$ items are prepared by the remaining parts of the QTG; $A_{n-s} := \text{QTG}_{n-s}$, so that the outer stage operates on the full state space with the same full oracle $O_n(y^{\text{inc}})$ used by the BA. The operators as defined in (5), restricted to the partial and full state spaces, are written D_s and D_n, respectively. We use r_{in} for the number of Grover iterations on the partial state space and $\tilde{r}$ for the number on the full state space. Gate costs are denoted $\mathcal{C}_{\text{IIF}}$ for a single IIF call and $\mathcal{C}$ for the running total of the whole NAA run. The per-shot cost $G_{\text{IIF}}(r_{\text{in}})$ of an inner validation and the per-step cost $G_{\text{outer}}(\tilde{r}, r_{\text{in}}^*)$ of an outer AA step are defined for each choice of q in Section 3.5. Termination criteria t_{IIF} and t_{GAS}, which may implement a gate budget, a convergence check, or a timelimit are discussed in Section 3.6.

Defining in more detail the action of the IIF, we identify the feasibility criterion on the partial state space as:

[8]From the fact that the algorithm cuts the binary search tree of the knapsack problem at this depth.

$$\sum_{m=1}^{s} w_m x_m \leq c. \tag{43}$$

The partial oracle marks all partial states $|x_{1:s}\rangle$ that are promising with respect to some binary criterion $q(y^{\text{inc}}, |x_{1:s}\rangle)$:

$$O_s^q(y^{\text{inc}})|x_{1:s}\rangle = \begin{cases} -|x_{1:s}\rangle & \text{if } q(y^{\text{inc}}, x_{1:s}) = 1 \\ |x_{1:s}\rangle & \text{otherwise} \end{cases}. \tag{44}$$

The concrete choice of q is deferred to Section 3.5. Given the feasibility and oracle definitions, the IIF applies AA on the register encoding s items, adopting the Grover iteration sampling of the QSearch routine described in Section 2.4. The goal, however, is not to measure a better incumbent but to identify a value of r_{in}^* that maximally amplifies promising partial states. We realize this by sampling r_{in} from $\{0, 1, ..., \lceil k \rceil - 1\}$, whose upper bound grows via $k \leftarrow \lambda k$ after each unsuccessful validation. For each candidate r_{in}, up to L validation shots are performed. If all L shots yield a state satisfying the criterion q, the IIF accepts r_{in} as r_{in}^* and returns. Otherwise, a new r_{in} is sampled from the enlarged range, and the process repeats until t_{IIF} (as described in Section 3.6.1) terminates it. A mathematical treatment of the upper and lower bounds of the success probabilities following from this approach can also be found in Section 3.6.1.

Algorithm 5: Inner Iteration Finder (IIF)

Require: Knapsack instance KS, depth s, incumbent x^{inc}, incumbent profit y^{inc}, criterion q
Require: Growth factor $\lambda > 1$, number of validation samples L, termination criterion t_{IIF}
Set $k \leftarrow 1$, $\mathcal{C}_{\text{IIF}} \leftarrow 0$
Set A_s^q to the partial state preparation on the first s items ▷ criterion-dependent
Set success $\leftarrow 0$
while not $t_{\text{IIF}}(\text{success}, L)$ **do**
  Sample r_{in} from $\{0, 1, ..., \lceil k \rceil - 1\}$
  Set success $\leftarrow 0$ and total $\leftarrow 0$
  repeat
    Apply $\left(-A_s^q D_s (A_s^q)^\dagger O_s^q(y^{\text{inc}})\right)^{r_{\text{in}}} A_s^q$
    Measure $\tilde{x}$ and $\tilde{y}$
    total $\leftarrow$ total $+ 1$
    $\mathcal{C}_{\text{IIF}} \leftarrow \mathcal{C}_{\text{IIF}} + G_{\text{IIF}}(r_{\text{in}})$
    if $q(y^{\text{inc}}, \tilde{x}) = 1$ **then**
      success $\leftarrow$ success $+ 1$
    end if
  until total $= L$ **or** success $\neq$ total
  Set $k \leftarrow \min(\lambda k, 2^s)$
end while
Set $r_{\text{in}}^* \leftarrow r_{\text{in}}$ ▷ accepted inner iteration count
return r_{in}^*, $\mathcal{C}_{\text{IIF}}$

For an accepted r^*_{in}, the operators acting on the partial state space are

$$\mathcal{A}_s^{(r^*_{\text{in}})} := \left(-A_s^q D_s (A_s^q)^\dagger O_s^q(y^{\text{inc}})\right)^{r^*_{\text{in}}} A_s^q. \tag{45}$$

Completing the QTG on the remaining $n-s$ items with A_{n-s} then yields the state preparation for the outer stage:

$$A_{\text{nested}} := A_{n-s}\mathcal{A}_s^{(r^*_{\text{in}})}. \tag{46}$$

The outer stage is now an ordinary GAS with state preparation A_{nested} and the full oracle $O_n(y^{\text{inc}})$ from (39). For each outer AA step with rotation count $\tilde{r}$, this applies

$$\left(-A_{\text{nested}} D_n A_{\text{nested}}^\dagger O_n(y^{\text{inc}})\right)^{\tilde{r}} A_{\text{nested}}. \tag{47}$$

Since O_n is identical to the oracle used by the BA, the difference between the two algorithms is confined to the state preparation where the BA uses A_n, the NAA uses A_{nested}. The algorithm stops at termination criterion t_{GAS} (as described in Section 3.6.2).

Algorithm 6: Nested Amplitude Amplification

Require: Knapsack instance KS with n items, depth s, criterion q, growth factor $\lambda > 1$
Require: Number of IIF validation samples L, termination criteria t_{IIF} and t_{GAS}
Get feasible incumbent x^{inc} and profit y^{inc} from greedy solution
Set $y^{\text{inc}} \leftarrow \sum_{i=1}^{n} p_i x_i^{\text{inc}}$
Set $k \leftarrow 1$, $\mathcal{C} \leftarrow 0$, $y_{\text{prev}} \leftarrow -\infty$
Set A_s^q to the partial state preparation, $A_{n-s} \leftarrow \text{QTG}_{n-s}$ ▷ criterion-dependent
Set $D_s \leftarrow 2|0\rangle\langle 0|^{\otimes s} - I^{\otimes s}$, $D_n \leftarrow 2|0\rangle\langle 0|^{\otimes n} - I^{\otimes n}$
while not $t_{\text{GAS}}(\mathcal{C})$ **do**
  if $y^{\text{inc}} \neq y_{\text{prev}}$ **then**
    $r^*_{\text{in}}, \mathcal{C}_{\text{IIF}} \leftarrow \text{IIF}(\text{KS}, s, x^{\text{inc}}, y^{\text{inc}}, q, \lambda, L, t_{\text{IIF}})$
    $\mathcal{C} \leftarrow \mathcal{C} + \mathcal{C}_{\text{IIF}}$
    Set $\mathcal{A}_s^{(r^*_{\text{in}})} \leftarrow \left(-A_s^q D_s (A_s^q)^\dagger O_s^q(y^{\text{inc}})\right)^{r^*_{\text{in}}} A_s^q$ ▷ partial state preparation
    Set $A_{\text{nested}} \leftarrow A_{n-s}\mathcal{A}_s^{(r^*_{\text{in}})}$ ▷ full-register state preparation
  end if
  $y_{\text{prev}} \leftarrow y^{\text{inc}}$
  $x^{\text{inc}}, y^{\text{inc}}, k, \tilde{r} \leftarrow \text{QSearch}(A_{\text{nested}}, O_n(y^{\text{inc}}), x^{\text{inc}}, y^{\text{inc}}, k, \lambda)$
  $\mathcal{C} \leftarrow \mathcal{C} + G_{\text{outer}}(\tilde{r}, r^*_{\text{in}})$
end while
return x^{inc}, y^{inc}

3.4 Gate Count of the Nested Approach

The gate cost of the NAA is made up of two parts: the cost the IIF accrues in order to settle on an inner iteration count r_{in}^*, and the cost of the outer GAS stage. We first give the per-shot and per-step costs, then aggregate them over a full run. As in Section 2.2, each Grover iteration applies its state preparation twice, so a stage with r Grover iterations applies its state preparation $2r + 1$ times.

A single IIF validation shot applies $\mathcal{A}_s^{(r_{\text{in}})}$ once (as in Algorithm 6) and measures. This requires the partial state preparation A_s^q a total of $2r_{\text{in}} + 1$ times and performs r_{in} partial Grover iterations, each contributing one diffusion $G_{=0}^s$ and one partial oracle to the gate count. Both the partial state preparation and the partial oracle depend on the criterion q: for the Profit-based Criterion (PBC) the partial state preparation reduces to QTG_s, whereas the Profit-and-Capacity-based Criterion (PCBC) requires additional gates. We therefore keep them abstract here, writing $G_{A_s^q}$ for the cost of A_s^q and $G_{O_s^q}$ for the cost of the partial oracle $O_s^q(y^{\text{inc}})$, both instantiated for each criterion q in Section 3.5. The per-shot cost is

$$G_{\text{IIF}}(r_{\text{in}}) = (2r_{\text{in}} + 1)G_{A_s^q} + r_{\text{in}}\Big(G_{=0}^s + G_{O_s^q}\Big), \tag{48}$$

which mirrors the per-step cost of the BA in (41), but only for the first s items and with a criterion-dependent state preparation and oracle.

The outer stage uses $A_{\text{nested}} = A_{n-s}\mathcal{A}_s^{(r_{\text{in}}^*)}$ as its state preparation. One application therefore completes the QTG on the remaining $n - s$ items and applies one IIF routine with the accepted count r_{in}^*, so its cost is

$$G_{A_{\text{nested}}}(r_{\text{in}}^*) = G_{\text{QTG}}^{n-s} + G_{\text{IIF}}(r_{\text{in}}^*). \tag{49}$$

The completion of the remaining $n - s$ items uses the standard QTG and is independent of q, hence the criterion dependence enters only through G_{IIF}. An outer AA step with $\tilde{r}$ Grover iterations applies A_{nested} a total of $2\tilde{r} + 1$ times and performs $\tilde{r}$ diffusion $G_{=0}^n$ and oracle $G_{>y^{\text{inc}}}^{|P|}$ (from Eqs. (39) and (40)) applications. Hence

$$G_{\text{outer}}(\tilde{r}, r_{\text{in}}^*) = (2\tilde{r} + 1)G_{A_{\text{nested}}}(r_{\text{in}}^*) + \tilde{r}\Big(G_{=0}^n + G_{>y^{\text{inc}}}^{|P|}\Big). \tag{50}$$

An execution of the NAA is scheduled around incumbent updates. Each update triggers a fresh IIF call, whose accepted count $r_{\mathrm{in}}^{*,(k)}$ is then used by the outer stage until the next update or until termination. Between the k-th and $(k+1)$-th update, the outer stage performs $N_{\mathrm{outer}}^{(k)}$ AA steps using the resulting A_{nested} as a state preparation. Let N_{updates} denote the total number of incumbent updates in the run (i.e. until the termination).

The IIF call triggered by the k-th incumbent update accumulates the per-shot cost of (48) over its $N_{\mathrm{IIF}}^{(k)}$ validation shots,

$$\mathcal{C}_{\mathrm{IIF}}^{(k)} = \sum_{i=1}^{N_{\mathrm{IIF}}^{(k)}} G_{\mathrm{IIF}}\left(r_{\mathrm{in},i}^{(k)}\right), \tag{51}$$

where $r_{\mathrm{in},i}^{(k)}$ is the candidate number of Grover iterations sampled at the i-th validation shot of the k-th incumbent update, and settles on an accepted count $r_{\mathrm{in}}^{*,(k)}$ (see Algorithm 5). The outer stage then accumulates the per-step cost over its $N_{\mathrm{outer}}^{(k)}$ AA steps,

$$\mathcal{C}_{\mathrm{outer}}^{(k)} = \sum_{t=1}^{N_{\mathrm{outer}}^{(k)}} G_{\mathrm{outer}}\left(\tilde{r}_t^{(k)}, r_{\mathrm{in}}^{*,(k)}\right), \tag{52}$$

where $\tilde{r}_t^{(k)}$ is the outer Grover iteration count sampled at step t of the k-th update. The cost associated with the k-th incumbent update is

$$\mathcal{C}^{(k)} = \mathcal{C}_{\mathrm{IIF}}^{(k)} + \mathcal{C}_{\mathrm{outer}}^{(k)}, \tag{53}$$

and the total nested cost is the sum over all incumbent updates,

$$\mathcal{C}_{\mathrm{nested}}^{q} = \sum_{k=1}^{N_{\mathrm{updates}}} \mathcal{C}^{(k)}. \tag{54}$$

The whole chain depends on the criterion q through A_s^q and $O_s^q(y^{\mathrm{inc}})$, for brevity this was left implicit until now and made explicit only on $\mathcal{C}_{\mathrm{nested}}^{q}$. The accumulator $\mathcal{C}$ in Algorithm 6 realizes (54) by incremental accumulation across incumbent updates.

3.5 Nested Amplification Criteria

The explanation of the NAA in Section 3.3 treats the partial oracle $O_s^q(y^{\text{inc}})$ in a general way, deferring the concrete choice of the criterion $q(y^{\text{inc}}, x_{1:s})$ to this section. We study two different choices of q, which we call the PBC and the PCBC. Generally, the goal of such criteria is to reject as many states as possible at depth s that cannot be completed to a marked state at depth n, while still accepting all states that can be completed to a marked state. This concentrates the probability mass on the promising states, which in turn allows a smaller number of Grover iterations $\tilde{r}$ to be used in the outer stage.

3.5.1 Profit-based Criterion (PBC)

This criterion only uses information about the profits of the partially assigned items, but not their weights. Instances for the PBC should therefore be sorted by descending profit before applying the algorithm, since placing the smallest profits last minimises the correction term $\sum_{m=s+1}^{n} p_m$ and thereby tightens the criterion.

The idea behind the PBC is to reject partial states $|x_{1:s}\rangle$ that cannot possibly be completed to a state $|x\rangle$ with a profit exceeding that of the incumbent y^{inc}, even if all remaining $n-s$ items were included (irrespective of capacity constraints). This is done by comparing the profit of the partial state $\sum_{m=1}^{s} p_m x_m$ to the incumbent profit y^{inc}. Formally, the PBC defines the criterion q as

$$q_{\text{PBC}}(y^{\text{inc}}, x_{1:s}) = \begin{cases} 1 \text{ if } \quad \sum_{m=1}^{s} p_m x_m > y^{\text{inc}} - \sum_{m=s+1}^{n} p_m \\ 0 \text{ otherwise.} \end{cases} \tag{55}$$

It follows from the definition that this criterion is loose: it accepts partial states that only lead to better solutions on the entire search space if the capacity constraint is violated. However, the criterion is simple to implement. In particular, the criterion is strong if many partial states produce profits so small, that not even by including all remaining items, the incumbent profit can be exceeded.

We now discuss the design of the oracle $O_s^{\text{PBC}}(y^{\text{inc}})$, which implements the PBC by marking partial states according to the criterion defined above. The oracle is implemented straightforwardly using the comparator primitive from Section 2.5.2. The state preparation on the first s items already allows us to access the cumulative profit of the partial state, which is why it suffices to mark all states that have a better partial cumulative profit

than the threshold $\tilde{y}^{\text{inc}} = y^{\text{inc}} - \sum_{i=s+1}^{n} p_i$, which is classically (and efficiently) computable at depth s.

Section 3.4 leaves the exact gate count for the oracle, $G_{O_s^q}$ and the partial state preparation $G_{A_s^q}$ to be defined later, since they depend on the choice of the criterion q. The gate count of the state preparation A_s^{PBC} is straightforwardly identified as the gate count of the QTG circuit on s qubits,

$$G_{A_s^{\text{PBC}}} = G_{\text{QTG}}^s, \tag{56}$$

since for the PBC we have $A_s^q = \text{QTG}_s$. As explained above, the oracle $O_s^{\text{PBC}}(y^{\text{inc}})$ can be implemented using the comparator circuit from Section 2.5.2, which leads to a gate count of

$$G_{O_s^{\text{PBC}}}(y^{\text{inc}}) = G_{>\tilde{y}^{\text{inc}}}^{|P|}, \tag{57}$$

with $\tilde{y}^{\text{inc}} = y^{\text{inc}} - \sum_{i=s+1}^{n} p_i$. In conjunction with (51), (52), (53) and (54) this fully defines the cost of the NAA using the PBC as a criterion for marked partial states.

3.5.2 Profit-and-Capacity-based Criterion (PCBC)

The PBC is simple to implement, but very loose. Not taking the capacity of the knapsack and the weights of the items into consideration when deciding which partial states are promising makes the PBC a weak filter. For this reason we introduce the PCBC. Since the oracle $O_s^{\text{PCBC}}(y^{\text{inc}})$ needs to take more into account than simply the cumulative profit of the partial state, the partial state preparation A_s^{PCBC} also needs to be more complex than the QTG circuit on s qubits.

The idea for the PCBC was originally motivated by classical BnB algorithms. In such algorithms, an Upper Bound (UB) to the obtainable objective value (profit in the case of the knapsack problem) is calculated at several depths. Branches of the search tree where the UB to the obtainable profit is lower than the incumbent profit are pruned, since they cannot lead to a better solution [6, 47]. Classical BnB algorithms form a solid base for the design of quantum algorithms, since they tend to be some of the most successful classical approaches to combinatorial optimization problems [48]. In the classical setting, they provide certificates of optimality and have seen substantial development, with comparative speedups of many orders of magnitude over a few decades [49]. In a quantum setting,

research has shown BnB to be a promising candidate for the emerging field of quantum combinatorial optimization [28]. Hence, the PCBC implements a similar idea, however only at one depth and in quantum superposition.

The idea behind the PCBC is to reject partial states $|x_{1:s}\rangle$ that cannot possibly improve upon the incumbent profit y^{inc}, even if the remaining capacity $C = c - \sum_{m=1}^{s} w_m x_m$ were fully filled with the best remaining item, judged by efficiency $\rho = \frac{p}{w}$. This resembles a quantised version of the fractional relaxation bound introduced by Dantzig [50], which is the standard UB used to prune the knapsack search tree in classical BnB solvers [11]. The Dantzig bound relaxes the integrality constraint of the knapsack problem. After sorting the items by descending efficiency $\rho_m = \frac{p_m}{w_m}$, it greedily includes whole items until the next item no longer fits, and then admits a fraction of that item to fill the capacity exactly. Because fractional inclusion can only increase the attainable profit, the resulting value is a valid UB on the integer optimum. The PCBC adopts a simplified, looser version of this bound: rather than greedily filling with successive items, it fills the entire remaining capacity fractionally with the single most efficient remaining item, as we only compute the UB at one depth s. Among the remaining $n - s$ items, we can identify the best item

$$m^* = \text{argmax}_{m=s+1,\ldots,n}(\rho_m) \tag{58}$$

and check if the profit of the partial state $\sum_{m=1}^{s} p_m x_m$ plus the profit obtained by fractionally filling the remaining capacity with the best item, with efficiency $\rho_{m^*} = \frac{p_{m^*}}{w_{m^*}}$, exceeds the incumbent profit y^{inc}. This leads to the following definition of the criterion q for the PCBC:

$$q_{\text{PCBC}}(y^{\text{inc}}, x_{1:s}) = \begin{cases} 1 \text{ if } \quad \sum_{m=1}^{s} p_m x_m + \rho_{m^*}\left(c - \sum_{m=1}^{s} w_m x_m\right) > y^{\text{inc}} \\ 0 \text{ otherwise.} \end{cases} \tag{59}$$

or expressed differently,

$$q_{\text{PCBC}}(y^{\text{inc}}, x_{1:s}) = \begin{cases} 1 \text{ if } \quad \sum_{m=1}^{s} (p_m - \rho_{m^*} w_m) x_m > y^{\text{inc}} - \rho_{m^*} c \\ 0 \text{ otherwise.} \end{cases} \tag{60}$$

The criterion is evidently looser if the set $\{\rho_m : m > s\}$ is dominated by an outlier, as this would reduce the modified threshold $y^{\text{inc}} - \rho_{m^*}c$. Comparing the two criteria, the PCBC rejects more states than the PBC (i.e., is tighter) if:

$$\sum_{m=s+1}^{n} p_m > \rho_{m^*}\left(c - \sum_{m=1}^{s} w_m x_m\right). \tag{61}$$

As with the PBC, we assume the instance is sorted before applying the algorithm, here by descending efficiency $\rho_m = \frac{p_m}{w_m}$. With this ordering the best remaining item is always $m^* = s + 1$.

In order to implement (59), we need to define both the partial state preparation A_s^{PCBC} and the oracle $O_s^{\text{PCBC}}(y^{\text{inc}})$. In particular, the circuit to implement $q_{\text{PCBC}}(y^{\text{inc}}, x_{1:s})$ requires an extra register $|\text{UB}\rangle$, that holds the UB for the profit if the best-efficiency was fractionally included filling the remaining capacity. Following the QTG, we have access to the profit register $|P\rangle$ storing the cumulative profit of the partial state, and the remaining capacity register $|C\rangle$. Given some partial state's profit P and remaining capacity C, the partial state should only be accepted by (59) if:

$$P + \left(\frac{p_{m^*}}{w_{m^*}}\right)C > y^{\text{inc}}, \tag{62}$$

where $\frac{p_{m^*}}{w_{m^*}} = \rho_{m^*}$ is the efficiency of the best remaining item. Of course ρ_{m^*} can generally be a fractional number, which can make quantum arithmetic more challenging [51]. To avoid this complication, we instead encode the criterion (59) for a partial assignment as:

$$w_{m^*}P + p_{m^*}C > w_{m^*}y^{\text{inc}}. \tag{63}$$

By guaranteeing all profits and weights are integers, fractional arithmetic is avoided, at the cost of $\log_2(w_{m^*})$ extra qubits. In order to realize (63), the state preparation A_s^{PCBC} acts on an additional register $|\text{UB}\rangle$ that holds the value $w_{m^*}P + p_{m^*}C$. The cumulative profit is bounded by $P = \sum_{m=1}^{s} p_m x_m \leq \sum_{m=1}^{s} p_m$, the remaining capacity by $C \leq c$, and the best-item coefficients by the global maxima $w_{m^*} \leq w_{\max} \coloneqq \max_m w_m$ and $p_{m^*} \leq p_{\max} \coloneqq \max_m p_m$. Hence the register has worst-case size

$$|\mathrm{UB}| = \left\lfloor \log_2 \left(w_{\max} \sum_{m=1}^{s} p_m + p_{\max} c \right) \right\rfloor + 1. \tag{64}$$

The register $|\mathrm{UB}\rangle$ is the only additional register the NAA requires over the BA; the widths of all other registers, as well as the ancillas needed by the comparator and multi-controlled gates, are unchanged from [20]. In order to store the value $P + \left(\frac{p_{m^*}}{w_{m^*}}\right) C$ in UB, the state preparation begins by the normal QTG steps (capacity controlled Hadamard gate, decision controlled capacity and profit register updates) as shown in Figure 2. Following these steps, two additions have to be performed: that of $w_{m^*} P$ and that of $p_{m^*} C$. Principles developed in Section 2.5.1 now come in handy. A_s^{PCBC} hence performs the partial QTG_s first and is then extended by $\mathrm{ADD}_{w_{m^*} P}$ and $\mathrm{ADD}_{p_{m^*} C}$. The resulting circuit is shown in Figure 4.

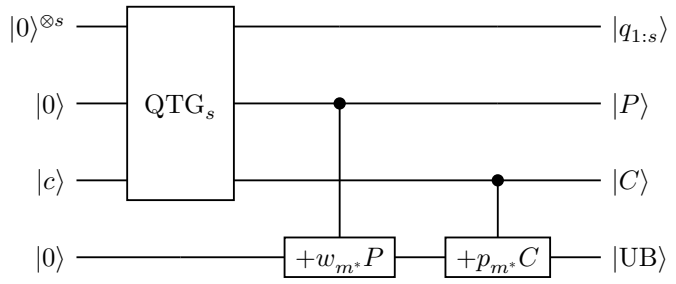


Figure 4: Quantum circuit of the state preparation A_s^{PCBC} for the Profit-and-Capacity-based Criterion (PCBC) at depth s. The block QTG_s denotes the partial Quantum Tree Generator (QTG) on the first s items, whose structure is described in Section 2.6.1 and shown in Figure 2, producing the cumulative profit register $|P\rangle$ and the remaining capacity register $|C\rangle$. Two subsequent additions first add $w_{m^*} P$ controlled on the values in the profit register, the second adds $p_{m^*} C$ controlled on the values in the remaining capacity register.

Having defined the action of A_s^{PCBC}, we now turn to the oracle $O_s^{\mathrm{PCBC}}(y^{\mathrm{inc}})$. The oracle marks all states that satisfy (63), which can be implemented by a simple comparator circuit from Section 2.5.2, very similar to the approach outlined in Section 3.5.1. The only difference here is the modified threshold $w_{m^*} y^{\mathrm{inc}}$, which is classically computable at depth s.

We now turn to the gate count. The two additions performed by A_s^{PCBC}, following (16) have gate counts:

$$G^{\mathrm{UB}}_{\mathrm{ADD}_{w_{m^*}P}} = G^{|\mathrm{UB}|}_{\mathrm{QFT}} + \sum_{i=1}^{|P|} \max(0, 3(|\mathrm{UB}| - \mathrm{LSO}(w_{m^*}) - i) + 4), \tag{65}$$

and

$$G^{\mathrm{UB}}_{\mathrm{ADD}_{p_{m^*}C}} = G^{|\mathrm{UB}|}_{\mathrm{QFT}} + \sum_{i=1}^{|C|} \max(0, 3(|\mathrm{UB}| - \mathrm{LSO}(p_{m^*}) - i) + 4). \tag{66}$$

At first glance, this might seem to disagree with the gate count given in (16). However, since both additions act on the same register, each addition only requires one QFT instead of two, as we can keep $|\mathrm{UB}\rangle$ in Fourier space. Secondly, the argument from (32) and [20] plays a vital role. In order to consistently apply the convention of optimising for parallel executability, each addition is implemented by copying and uncopying the register it is controlled on, namely $|P\rangle$ and $|C\rangle$, respectively. Applying (32) yields (65) and (66). The entire state preparation A_s^{PCBC} hence has a gate count of

$$G_{A_s^{\mathrm{PCBC}}} = G^s_{\mathrm{QTG}} + G^{\mathrm{UB}}_{\mathrm{ADD}_{w_{m^*}P}} + G^{\mathrm{UB}}_{\mathrm{ADD}_{p_{m^*}C}}. \tag{67}$$

Having defined the cost of A_s^{PCBC}, we now turn to the oracle $O_s^{\mathrm{PCBC}}(y^{\mathrm{inc}})$. As constructed above, the oracle hence has a gate count of

$$G_{O_s^{\mathrm{PCBC}}(y^{\mathrm{inc}})} = G^{|\mathrm{UB}|}_{>w_{m^*}y^{\mathrm{inc}}}. \tag{68}$$

Eq. (67) and (68), applied to (48) yields the total cost of NAA using the PCBC criterion.

3.6 Termination Criteria

Finally, we move to the discussion of the termination criteria. Section 3.3 introduces the need for two different types of termination criteria: t_{IIF} and t_{GAS}. The former is used to identify a condition at which the IIF settles on number of inner Grover iterations r^*_{in} to return, while the latter is used as a condition to stop the execution of the entire algorithm itself. Crucially, this termination criterion applies to both the BA and the NAA, whereas the t_{IIF} only applies to the IIF and thereby the NAA. In this section, we discuss both termination criteria in detail.

3.6.1 t_{IIF}

As described in Algorithm 5, the termination condition for the IIF is a function of the number of successful measurements of marked partial states, denoted here by success, and a measurement threshold L. Concretely, the IIF terminates once L measurements in a row land in the marked partial subspace. The underlying idea relies on the probability mass p of the marked partial subspace. Measuring a marked state repeatedly without fail gives high confidence that p is large, a large p means the partial amplification has concentrated the probability mass well, and good amplification can only occur when the sampled number of inner Grover iterations r_{in} is close to the optimum. Repeated successful measurements are therefore a reliable proxy for r_{in} sitting near this optimal value. Should any single shot fail before L successes are reached, the counter success is reset and a fresh r_{in} is sampled from an enlarged range, as detailed in Algorithm 5. The criterion can be expressed as:

$$t_{\text{IIF}}(\text{success}, L) = \begin{cases} \text{True if} \quad \text{success} = L \\ \text{False otherwise.} \end{cases} \tag{69}$$

The criterion is of a classical nature (since the shot outcomes of the measurements are classical), which is why we need not bother about a quantum circuit implementation. However, we discuss the statistical properties of this termination criterion in the remainder of this section.

Less strict implementations of this termination criterion are possible. Allowing a small but non-zero number of failures in the L measurements, is another possible implementation. However, the ultimate goal of this approach is to minimise the number of measurements taken, to reduce the cost of the IIF, while still maintaining high confidence that the inner Grover iterations r_{in} are close to the optimal value r_{in}^*. We have found the implementation of (69) to be a good balance between these two goals, and defer the investigation of other such criteria to future work.

Naturally, we would like to quantify exactly how confident we can be that the number of inner Grover iterations is close to the optimal value. Regarding the sampling that occurs with each measurement in the IIF as a binomial sampling procedure, we can identify confidence bounds on the probability mass in the marked partial subspace using the Clopper-Pearson confidence interval [52]. The probability of whether we measure a

marked partial state depends on the probability mass p in the marked subspace, which in turn depends on r_{in}:

$$p = \sum_{x \in \text{ marked}} |\langle x | \mathcal{A}_s^{r_{\text{in}}} | 0 \rangle|^2. \tag{70}$$

This state is prepared anew before each measurement, which motivates the treatment as a binomial process. Following the Clopper-Pearson confidence interval, we can set bounds for the probability

$$p_{\text{lower}} \leq p \leq p_{\text{upper}}, \tag{71}$$

of which we are $1 - \alpha$ confident (where α is the significance level). Following (69), i.e. the number of successes equals the number of measurements L, the bounds can be expressed analytically [53]:

$$p_{\text{lower}} = \left(\frac{\alpha}{2}\right)^{\frac{1}{L}} \quad \text{and} \quad p_{\text{upper}} = 1. \tag{72}$$

For our empirically chosen value of $L = 5$, Table 1 lists the resulting confidence intervals for several confidence levels.

$1 - \alpha$	p_{lower}	p_{upper}
80%	0.631	1
90%	0.549	1
95%	0.478	1
99%	0.347	1

Table 1: Clopper-Pearson confidence intervals for different significance levels with $L = 5$ measurements at the Inner Iteration Finder (IIF) level.

3.6.2 t_{GAS}

Lastly, we discuss the termination criterion t_{GAS}. It is used both in the BA and the NAA, and determines at which point the entire algorithm should terminate. Several approaches are possible, like termination upon convergence, a fixed number of iterations or some budget in terms of gate count [19]. In this thesis, we focus on the latter. Given some budget $\mathcal{B}$ in terms of gates (in the gate counting framework of [20]), we can terminate the algorithm once the accumulated gate count exceeds this budget. Formally, we can express this criterion as:

$$t_{\text{GAS}}(\mathcal{B}, \mathcal{C}) = \begin{cases} \text{True if} \quad \mathcal{C} > \mathcal{B} \\ \text{False otherwise.} \end{cases} \tag{73}$$

where $\mathcal{C}$ in our case would be the cost $\mathcal{C}_{\text{baseline}}$ or $\mathcal{C}_{\text{nested}}$, implemented using a cumulative counter as outlined in Algorithm 6.

4 Implementation and Simulation Approach

Chapter 3 explains the theoretical contribution of this thesis. In order to evaluate whether the NAA shows any advantage over the BA, several things are needed for benchmarking: instances to benchmark on, a way to classically benchmark without having to employ quantum hardware and metrics to compare the results. This chapter serves to introduce all of these things and concludes with the summary of the implementation pipeline.

4.1 Instance Generator

Knapsack instances can be categorized by different parameters, such as the number of items, capacity tightness, the distribution of weights and profits, and many more [25]. For the purpose of this thesis, we chiefly divide datasets by the correlation between their weights and profits. Pisinger [7] provides an instance generator for the binary knapsack problem, that considers exactly this parameter. We use this generator to create our datasets. Each instance can be instantiated by the following parameters:

- Number of items n,
- Range r, s.t. weights are sampled uniformly from $[1, r]$,
- Tightness of the knapsack S, where a larger S yields a smaller capacity and thus a tighter knapsack,
- An instance number i, which both influences the capacity and acts as a seed in pseudo-random sampling,
- Correlation type between weights and profits
  - Uncorrelated: profits are sampled uniformly from $[1, r]$ and are independent of the weights,
  - Weakly correlated: profits are sampled uniformly from $\left[\max\left(1, w_m - \frac{r}{10}\right), w_m + \frac{r}{10}\right]$ for each weight w_m,
  - Strongly correlated: profits are set to $p_m = w_m + 10$ for each weight w_m.

The instance is then sampled according to the parameters and receives capacity

$$c = \max\left(\left\lfloor \frac{i}{S+1} \sum_{m=1}^{n} w_m \right\rfloor, r+1\right). \tag{74}$$

For benchmarking purposes, we create instances in bulk. This exposes two degenerate regimes of (74). If the capacity gets set to $r + 1$, instances collapse onto the same capacity, producing exact or approximate instance duplicates. When i grows too large, all items fit and the instance becomes trivial. Since we reuse instance numbers across different parameters S, r and n, care must be taken to avoid both cases. We briefly derive conditions to avoid them.

Since weights are sampled uniformly from $[1, r]$, the expected total weight of an instance is

$$\mathbb{E}\left[\sum_{m=1}^{n} w_m\right] = n\frac{r+1}{2}. \tag{75}$$

The whole item set fits, rendering the instance trivial, once $c = \frac{i}{S+1}\sum_{m=1}^{n} w_m \geq \sum_{m=1}^{n} w_m$, i.e.

$$i \geq S + 1. \tag{76}$$

The capacity lower bound $c = r + 1$ in (74) is triggered when $\frac{i}{S+1}\sum_{m=1}^{n} w_m \leq r$. Substituting (75), avoiding duplicates requires

$$in(r + 1) > 2r(S + 1). \tag{77}$$

Combining (76) and (77) yields the valid range for the instance number i:

$$\frac{2r(S+1)}{n(r+1)} < i < S + 1. \tag{78}$$

We produce benchmarking instances in this range. Because (77) replaces the realized weight sum with its expectation (75), the lower bound holds only on average.

Furthermore, this thesis focuses on uncorrelated and weakly correlated instances, the strongly correlated type is listed above for completeness but is not used in our benchmarks. Uncorrelated instances tend to be easier to solve classically, because very efficient items (that are most likely part of the solution) can be easily identified. As discussed briefly in Section 2.1 and demonstrated in [9], correlated instances are classically harder, as items' efficiencies are similar. Moreover it has been shown, that instances with capacities that are not too loose and not too tight are harder to solve, as they have a larger number

of feasible solutions [9]. Our benchmarking datasets address instance tightness as well as correlation.

Concretely, both benchmark datasets share the same parameter grid, so that any difference between them is attributable to the correlation type alone. We fix $r = 400$ and sweep $n \in [10, 35]$, choosing for each n a tightness S and enumerating i over the admissible range of (78). Sweeping i at fixed S sweeps the capacity from the duplicate-avoiding lower bound up to the triviality threshold, so S controls the resolution of that sweep and therefore varies with n. The upper limit $n = 35$ is imposed by the classical simulation. Instances where the greedy solution is already optimal are of no interest to our optimization and are subsequently filtered out. The benchmark instances are available at https://github.com/LaurinDemmler/Masterthesis_Nested_Amplitude_Amplification.

4.2 Classical Amplitude Tracking

Scaling of quantum hardware has been impressive at providing a larger number of less noisy qubits [54–57]. This has led to a larger optimization problems being tractable on hardware [58, 59]. Regardless, AA amplification circuits tend to be deep, and for knapsack instances of interesting sizes require larger numbers of qubits than currently available. In addition, execution would ideally require a large number of error-corrected qubits, which are not yet available at scale.

Classical simulation tools for quantum circuits have likewise improved in quality and their ability to simulate complex systems. In particular, approaches like Matrix Product States (MPS) [60, 61] or the application of high-performance computing [62] have shown promise in quantifying behaviour of quantum circuits and systems not yet available on hardware. However, even these approaches are naturally limited by the exponential scaling of quantum systems and do not take problem specific structures into account. In this thesis, we exploit the structure of the NAA as well as make use of the sophisticated nature of modern classical solvers. In a nutshell, we avoid the tracking of amplitudes of states that are not marked (or infeasible) in the algorithm and thereby reduce the number of tracked elements in the otherwise exponentially large state vector. Moreover, this approach allows the study of the NAA in a non-worst-case asymptotic setting, which can be more representative of the performance of the algorithm in practice.[20, 23, 63]

In order to track relevant amplitudes with respect to our incumbent y^{inc} we first compute two different sets of marked, feasible states, $S_s^q(y^{\text{inc}})$ and $S_n(y^{\text{inc}})$. To this end we employ gurobipy-12.0.3 [64]. $S_s^q(y^{\text{inc}})$ and $S_n(y^{\text{inc}})$ are defined as:

$$S_n := S_n(y^{\text{inc}}) = \left\{ x \in \{0,1\}^n \mid \sum_{m=1}^{n} p_m x_m > y^{\text{inc}} \wedge \sum_{m=1}^{n} w_m x_m \leq c \right\} \tag{79}$$

and

$$S_s^q := S_s^q(y^{\text{inc}}, s) = \left\{ x \in \{0,1\}^s \mid q(y^{\text{inc}}, x) = 1 \wedge \sum_{m=1}^{s} w_m x_m \leq c \right\}, \tag{80}$$

where $q(y^{\text{inc}}, x)$ is a nested amplification criterion defined in Section 3.5. The set $S_s^q(y^{\text{inc}}, s)$ contains all partial states satisfying the nested criterion $q(y^{\text{inc}}, x)$ and are partially feasible. $S_n(y^{\text{inc}})$ contains all feasible, marked states on the full search space. Computing these sets using a classical solver is the limiting factor in the simulation of the NAA. Due to the quality of modern solvers however, instance sizes can be relatively large.

Ultimately, in Algorithm 6 for the NAA and Algorithm 2 for the BA, we are interested in simulating measurements of the quantum state produced by sampling from the states (and their amplitudes) in $S_n(y^{\text{inc}})$. Since only marked states affect the update loop in GAS (Algorithm 2), sampling a state that is not marked simply happens with amplitude

$$1 - \sum_{x' \in S_n(y^{\text{inc}})} |\beta_{x'}|^2, \tag{81}$$

where $\beta_{x'}$ is the amplitude of state x'. Their exact amplitude distribution can be neglected as the only information necessary for the algorithm to be benchmarked is whether or not an unmarked state is measured. Of course the amplitude distribution of marked states needs to be tracked, as a measurement of the marked subspace updates the incumbent. We therefore proceed by outlining how to track these amplitudes.

To distinguish between states in the partial or full search space, we adopt the following notation: α_x denotes the amplitude of a partial state $x \in S_s^q$, while $\beta_{x'}$ denotes the amplitude of a state $x' \in S_n$ on all decision variables.

Section 2.6 describes the effect of the QTG on some state $|x\rangle$, given a reference solution x^{ref} and bias b in detail. In our setup, the reference solution is the incumbent x^{inc} and we are interested in computing the amplitude of state $|x\rangle$, with current amplitude α_x. Eq. (27) shows the action of the biased Hadamard gate on state, given that the capacity constraint is satisfied. The QTG applied on item m transforms the amplitude α_x as follows:

$$\alpha_x \to \alpha_x h_m(x, x_m^{\text{inc}}, b), \tag{82}$$

where

$$h_m(x, x_m^{\text{inc}}, b) = \begin{cases} \sqrt{\frac{1+b}{2+b}} & \text{if } x_m = x_m^{\text{inc}} \text{ and } w_m \leq c - \sum_{j=1}^{m-1} w_j x_j \\ \sqrt{\frac{1}{2+b}} & \text{if } x_m \neq x_m^{\text{inc}} \text{ and } w_m \leq c - \sum_{j=1}^{m-1} w_j x_j \\ 1 & \text{if } x_m = 0 \text{ and } w_m > c - \sum_{j=1}^{m-1} w_j x_j. \end{cases} \tag{83}$$

The case $w_m > c - \sum_{j=1}^{m-1} w_j x_j$ corresponds to the situation where item m no longer fits into the remaining capacity, so no superposition is created and the amplitude remains unsplit. With (82) and (83) we can now track the amplitude α_x of some partial state $|x\rangle \in S_s^q(y^{\text{inc}}, s)$ as:

$$\alpha_x = \prod_{m=1}^{s} h_m(x, x_m^{\text{inc}}, b). \tag{84}$$

Using the amplitudes in the marked subspace $S_s^q(y^{\text{inc}}, s)$ after the QTG on the first s items, it is possible to track them through the actions of AA in the IIF. AA reduces to a known multiplicative factor on amplitudes, given the number of Grover iterations r_{in} [20, 38]. The amplitude α_x hence transforms as:

$$\tilde{\alpha}_x = \alpha_x \left(\frac{\sin((2r_{\text{in}} + 1)\theta_s)}{\sin(\theta_s)} \right), \tag{85}$$

where

$$\theta_s = \arcsin\left(\sqrt{\sum_{x \in S_s^q(y^{\text{inc}}, s)} |\alpha_x|^2} \right). \tag{86}$$

In particular, the re-scaling factor $\frac{1}{\sin(\theta_s)}$ ensures consistency even for initially non-uniform amplitude distributions.

Following the IIF, NAA applies the QTG on the remaining $n-s$ items. Additionally we can observe, that for each $x' \in S_n$, we must necessarily find that $x'_{1:s} \in S_s^q(y^{\text{inc}}, s)$, since states marked w.r.t. to (39) are necessarily marked w.r.t. to (55) or (59). Calculating the amplitude $\tilde{\beta}_{x'}$ of a state $x' \in S_n(y^{\text{inc}})$ after the QTG on the remaining items is then given by:

$$\tilde{\beta}_{x'} = \tilde{\alpha}_{x'_{1:s}} \prod_{m=s+1}^{n} h_m(x', x_m^{\text{inc}}, b), \tag{87}$$

where $x'_{1:s}$ is the projection of x' onto the first s items. Finally, we can track the amplitudes $\beta_{x'}$ of states in $S_n(y^{\text{inc}})$ through the AA of the outer application layer of the NAA, using $\tilde{r}$ number of Grover iterations. The reasoning follows (87), and we find:

$$\beta_{x'} = \tilde{\beta}_{x'} \left(\frac{\sin((2\tilde{r}+1)\theta_n)}{\sin(\theta_n)} \right), \tag{88}$$

where

$$\theta_n = \arcsin\left(\sqrt{\sum_{x' \in S_n(y^{\text{inc}})} |\tilde{\beta}_{x'}|^2} \right). \tag{89}$$

We note that since knowing θ_s and θ_n is necessary to track amplitudes, the entire sets $S_s^q(y^{\text{inc}}, s)$ and $S_n(y^{\text{inc}})$ must be pre-computed.

With the set of amplitudes $\beta_{x'}$ of states in $S_n(y^{\text{inc}})$ we can now simulate the measurement of the quantum state produced by the NAA. In particular, the probability of measuring a marked state is

$$p_{\text{marked}} = \sum_{x' \in S_n(y^{\text{inc}})} |\beta_{x'}|^2. \tag{90}$$

Simulating a shot measurement is then done by sampling candidate $x' \in S_n(y^{\text{inc}})$ with probability $\frac{|\beta_{x'}|^2}{p_{\text{marked}}}$ and any unmarked state with probability $1 - p_{\text{marked}}$.

Tracking amplitudes for the BA is done analogously, with the difference that only the set $S_n(y^{\text{inc}})$ needs to be pre-computed. The procedure described in this chapter is hence used in the measurements in Algorithm 5 and Algorithm 6 as a means to circumvent the need for full quantum state simulation. Since we compute classically, all relevant parameters to calculate cost as in Section 3.4 can be saved at runtime.

4.3 Experimental Metrics

Evaluating the performance of the NAA requires two different ingredients: a notion of relative cost between two approaches and a notion of quality of solution. We address both in this section.

4.3.1 Gate Count Ratio

Given a run of both the NAA and the BA approach to termination, gate costs of running the algorithms can be computed as demonstrated in (42) and (54). To ensure a symmetric comparison, we define the relative cost of running the NAA with respect to the BA as:

$$\mathcal{C}^q_{\text{rel}} = \log_2\left(\frac{\mathcal{C}_{\text{baseline}}}{\mathcal{C}^q_{\text{nested}}}\right). \tag{91}$$

A value larger than 0 indicates a cost advantage of the NAA.

4.3.2 Optimality Gap

Of course, a cost advantage is only meaningful if the quality of solution is not compromised. A natural measure of solution quality is the approximation ratio of some solution value y, with respect to the optimal solution value y^*:

$$\alpha(y) = \frac{y}{y^*}. \tag{92}$$

However, greedy solutions to knapsack instances are known to be already of high quality [65], which somewhat reduces the significance of the approximation ratio. Instead, we consider the optimality gap of a solution y with respect to the greedy solution y^{greedy}:

$$\eta(y) = \frac{\alpha(y) - \alpha\big(y^{\text{greedy}}\big)}{1 - \alpha(y^{\text{greedy}})}. \tag{93}$$

The optimality gap is a relative measure of quality, that is normalized to the quality of the greedy solution. A value of 0 indicates that the solution is of equal quality to the greedy solution, while a value of 1 indicates that the solution is optimal.

4.4 Implementation Pipeline

To conclude this chapter, we provide a high-level overview over the implementation pipeline. The code is available at https://github.com/LaurinDemmler/Masterthesis_Nested_Amplitude_Amplification.

The visual representation of the pipeline can be seen in Figure 5. Initially, knapsack instances are generated as described in Section 4.1. From those, a knapsack instance to simulate is chosen and the greedy value is computed, which is subsequently set as the incumbent values for the BA and NAA: $y^{\text{inc}}_{\text{baseline}}$ and $y^{\text{inc}}_{\text{nested}}$. Depending on the nested criterion (PCBC or PBC), the Gurobi-based classical solver now computes the sets S^q_s and S_n, as identified in Section 4.2. Both for the BA and the NAA a cumulative gate cost counter is initialised at 0. For the BA, we now track all amplitudes in S_n through the state preparation $A_n = \text{QTG}_n$. If the gate cost has not surpassed the budget (implemented through t_{GAS} from Section 3.6.2), the amplitudes are further tracked through the AA. As described in detail in Section 4.2, a candidate solution is sampled, if the cumulative gate cost does not surpass the budget. A similar approach can be seen in the NAA branch of the implementation. We start by tracking amplitudes in S^q_s through the state preparation A^q_s. If the budget is not surpassed, we enter the IIF, which returns the number of inner Grover iterations r^*_{in} and the amplitudes in S^q_s. The IIF itself uses the mechanism described in Section 3.3 to settle on the number of inner Grover iterations. The amplitudes in S_n are then tracked through the remaining state preparation QTG_{n-s}, using the amplitudes from S^q_s through (87). The amplitudes are then tracked through the outer AA, if the budget allows. As for BA, a candidate solution is sampled. After sampling, the incumbent is updated if the candidate solution is better than the incumbent. If that is the case, S^q_s and S_n are recomputed to start the next iteration. If at any point in the pipeline the gate cost surpasses the budget, the algorithm in that branch terminates, parameters of the run and metrics are computed and stored as described in Section 4.3.

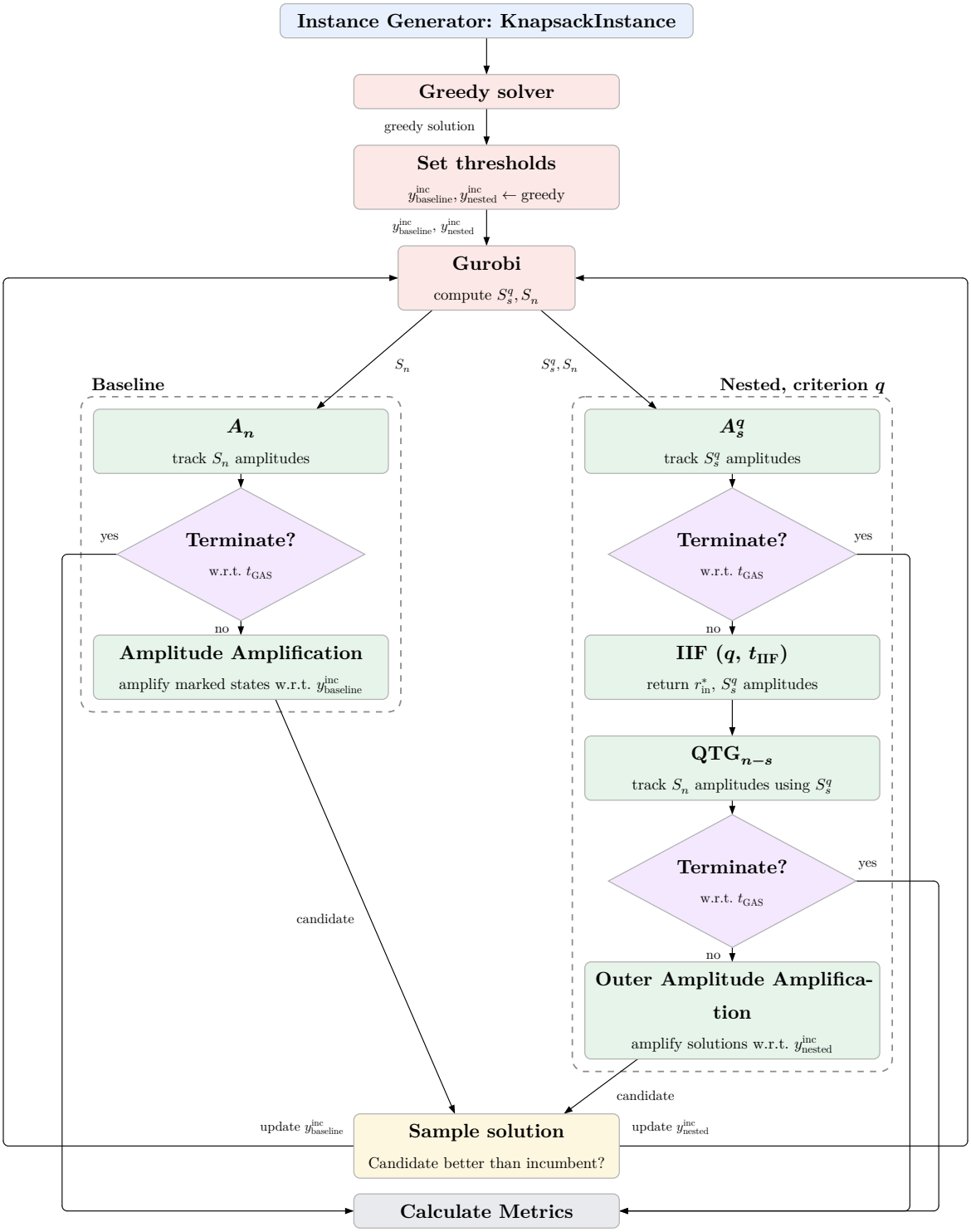


Figure 5: Implementation pipeline for the generation of results, benchmarking Nested Amplitude Amplification (NAA) against Baseline Approach (BA). After the knapsack instance is generated, the greedy solution is set as the incumbent for both approaches. Tracking amplitudes in S_s^q and S_n through the approaches is done by applying the classical transformations as described in Section 4.2. The termination criterion for budget, t_{GAS} is checked continuously to avoid exceeding the budget. Once the algorithm is terminated, metrics from Section 4.3 are stored.

5 Results

In this chapter, we examine the performance of the NAA in comparison to the BA from [20]. All sections use the implementation from Figure 5 and the two datasets specified in Section 4.1: one consisting of uncorrelated instances, and one consisting of weakly correlated instances.

In Section 5.1 we briefly justify the use of the bias $b = \frac{n}{4}$ for the BA, and in Section 5.2 we examine the effect of varying the bias on the performance of both approaches. In Section 5.3, we more closely examine the optimal number of inner iterations the IIF settles on, while Section 5.4 reports on the general difficulty of predicting an optimal depth s for the NAA. In Section 5.5 we compare the approaches under a fixed budget, first with empirically determined biases and finally in a scenario where all biases are 0 in Section 5.6.

5.1 Average Hamming Weight

Section 2.6.4 examines the importance of choosing sensible bias values for the convergence of GAS. In particular, Wilkening et al. [20] report empirically (on instances of up to size 600) that the average Hamming distance between the optimal solution and the reference solutions, $\Delta\big(x^{\text{opt}}, x^{\text{ref}}\big)$, is 4, which motivates their choice of $b = \frac{n}{4}$. To assess whether this bias is sensible for our datasets, we exploit the fact that, on average, $\Delta\big(x^{\text{opt}}, x^{\text{ref}}\big)$ is roughly upper bounded by the Hamming distance between the greedy solution and the optimum, $\Delta\big(x^{\text{opt}}, x^{\text{greedy}}\big)$. This bound does not hold strictly, since proximity in value does not guarantee proximity in the solution space, but it serves as a useful proxy. To this end, Figure 6 reports $\Delta\big(x^{\text{opt}}, x^{\text{greedy}}\big)$ for both the uncorrelated and the weakly correlated dataset over knapsack sizes up to 35 items, restricted to instances where greedy is not already optimal.

We recognize two trends. First, the weakly correlated instances exhibit a consistently higher mean Hamming distance (3.83) than the uncorrelated instances (2.89). The weakly correlated value lies close to the distance of 4 reported by [20], so $b = \frac{n}{4}$ appears appropriate here, whereas the uncorrelated instances would suggest a bias closer to $b = \frac{n}{3}$. Second, the Hamming distance grows with the knapsack size, which is expected as the solution space itself grows.

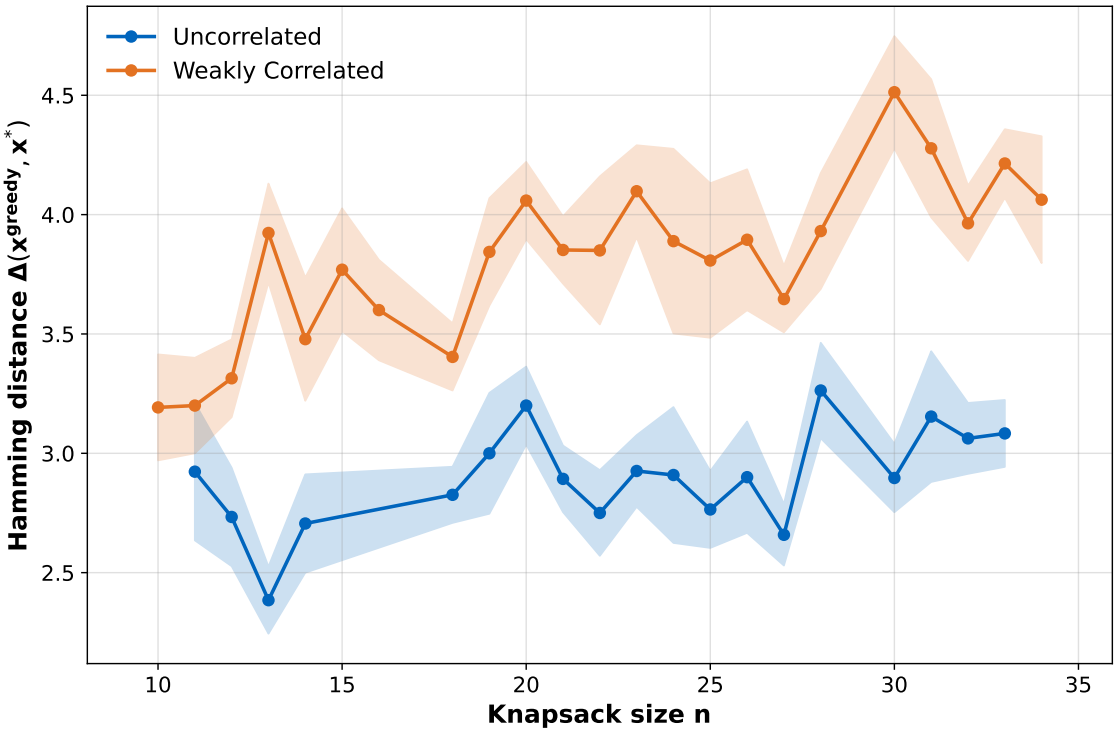


Figure 6: Hamming distance $\Delta\left(x^{\text{opt}}, x^{\text{greedy}}\right)$ between the greedy and optimal solution versus the number of items n for uncorrelated and weakly correlated instances. The x-axis is the instance size n and the y-axis is the number of differing bits. Weakly correlated instances have the larger average Hamming distance.

These results must be interpreted with care. Precomputing optimal biases by solving the instances of interest to optimality defeats the purpose of using GAS as an optimisation routine, so a bias must never depend on quantities that are only available after the fact. This does not rule out biases that depend on instance features which are classically cheap to obtain. The correlation type in particular can be estimated from the sample correlation between weights and profits, so a bias that differs between correlation classes remains admissible. What we do avoid is a bias tuned per instance. Since Figure 6 roughly agrees with [20], we adopt their choice of $b = \frac{n}{4}$ as the bias baseline for the BA throughout this thesis, primarily to allow for a more direct comparison between BA and NAA.

5.2 Bias Sweep

Section 3.3 describes the NAA. In particular, no matter the partial amplification criterion $q(y^{\text{inc}}, x_{1:s})$, the QTG is applied to the first s variables (and for the PCBC other gates follow to complete the partial state preparation A_s^q) and later applied to the remaining

$n - s$ variables. This means, that the NAA has two different bias parameters as degrees of freedom, whereas the BA only has one. The choice for the bias parameter for the BA has been discussed in Section 5.1. In this section, we identify promising bias values for the QTG on the first s variables, b_s^q, and the remaining $n - s$ variables, b_{n-s}^q, for both the PCBC and the PBC. To this end, we perform a bias sweep over the two parameters, where we vary b_s^q and b_{n-s}^q independently. The results are reported in Figure 7, Figure 8, Figure 9 and Figure 10.

All figures in this section use the Capacity to Weight Ratio (CWR) on the x-axis, to group knapsack instances of similar tightness (and therefore potentially difficulty [7]). A knapsack instance's CWR is defined as:

$$\text{CWR} = \frac{c}{\sum_{m=1}^{n} w_m}, \tag{94}$$

where a CWR > 1 yields a trivial instance with all items fitting in the knapsack.

All bias sweep experiments follow the same setup. We execute the pipeline from Figure 5, but stop the iterative improvement of GAS after the greedy solution is improved upon once, for reasons of computational load. This restriction potentially overpenalizes the NAA, as the marked subspace is largest when only improving the greedy solution. It serves, however, as a computationally tractable proxy for the full optimisation. Moreover, we sweep the depth s across the entire range of $[1, n-1]$ and choose the best depth (w.r.t. Eq. (91)) for each instance. This can be understood as a best-case scenario for the NAA, where the optimal depth s per instance is chosen. We consistently compare against the BA with bias $b = \frac{n}{4}$, as justified in Section 5.1, and compute the gate count ratio from (91).

We note that all bias values identified in this section are at least partially dependent on the instance sets used. Whether the same values remain favourable for substantially larger n, and how the two biases should scale relative to one another and relative to the chosen depth s, is left to future work. A promising direction here would be a depth aware bias, in which the bias parameter itself depends on the item m to be processed.

5.2.1 Bias Sweep for the Profit-based Criterion (PBC)

First, we set out to determine favourable bias parameters for the PBC. Figure 7 shows the result for uncorrelated instances, while Figure 8 shows the result for weakly correlated

instances. Both graphs show the best 5 bias combinations for b_s^{PBC} and b_{n-s}^{PBC} in color, while worse combinations are greyed out.

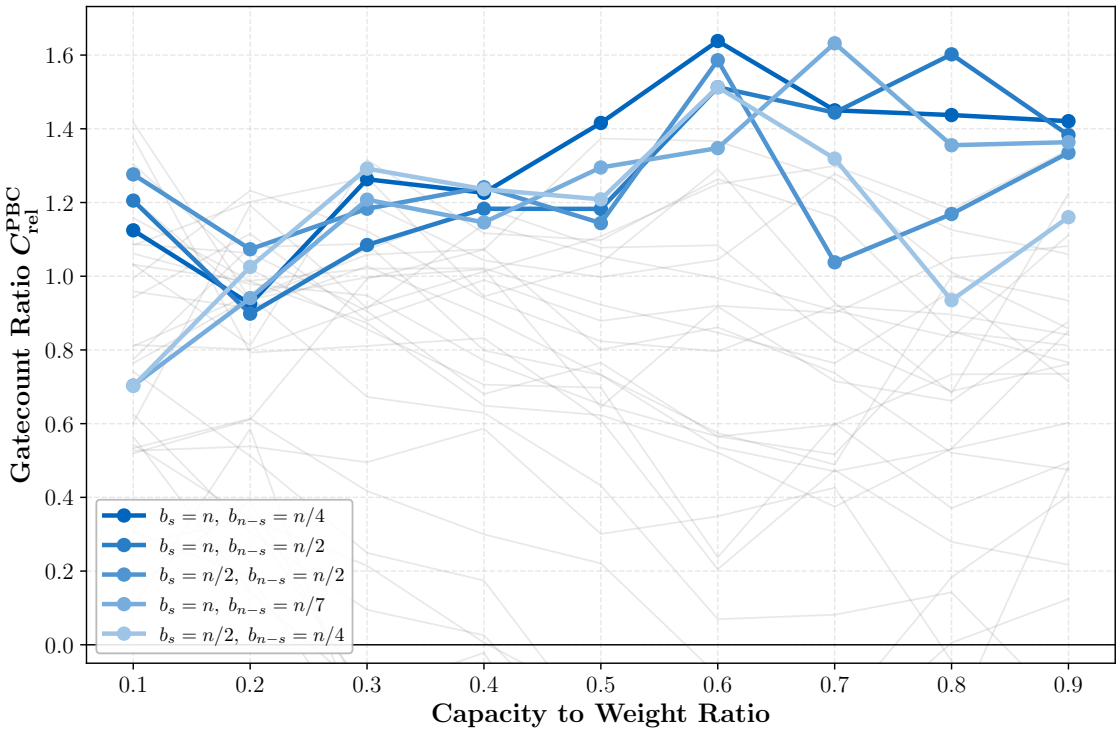


Figure 7: Bias sweep for the Profit-based Criterion (PBC) on uncorrelated knapsack instances. The x-axis is the capacity-to-weight ratio CWR, which groups instances by capacity-tightness, and the y-axis reports the mean gate count ratio (from Eq. (91)) relative to the Baseline Approach (BA) for each bias pair. Colored curves mark the five best bias pairs. The best average performance is obtained at $b_s^{\mathrm{PBC}} = n$ and $b_{n-s}^{\mathrm{PBC}} = \frac{n}{4}$.

Figure 7 shows that for the PBC criterion on uncorrelated instances the best bias combination on average is $b_s^{\mathrm{PBC}} = n$ and $b_{n-s}^{\mathrm{PBC}} = \frac{n}{4}$. Figure 8 shows that for the PBC criterion on weakly correlated instances the best bias combination on average is also $b_s^{\mathrm{PBC}} = n$ and $b_{n-s}^{\mathrm{PBC}} = \frac{n}{4}$. While the next best performing combinations in both examples differ slightly, we observe two trends. First, the value $b_{n-s}^{\mathrm{PBC}} = \frac{n}{4}$ is a good choice across the leading combinations of both instance classes. Second, the leading combinations within each class perform very similarly to each other on average, with deviations of at most 12% in the gate count ratio.

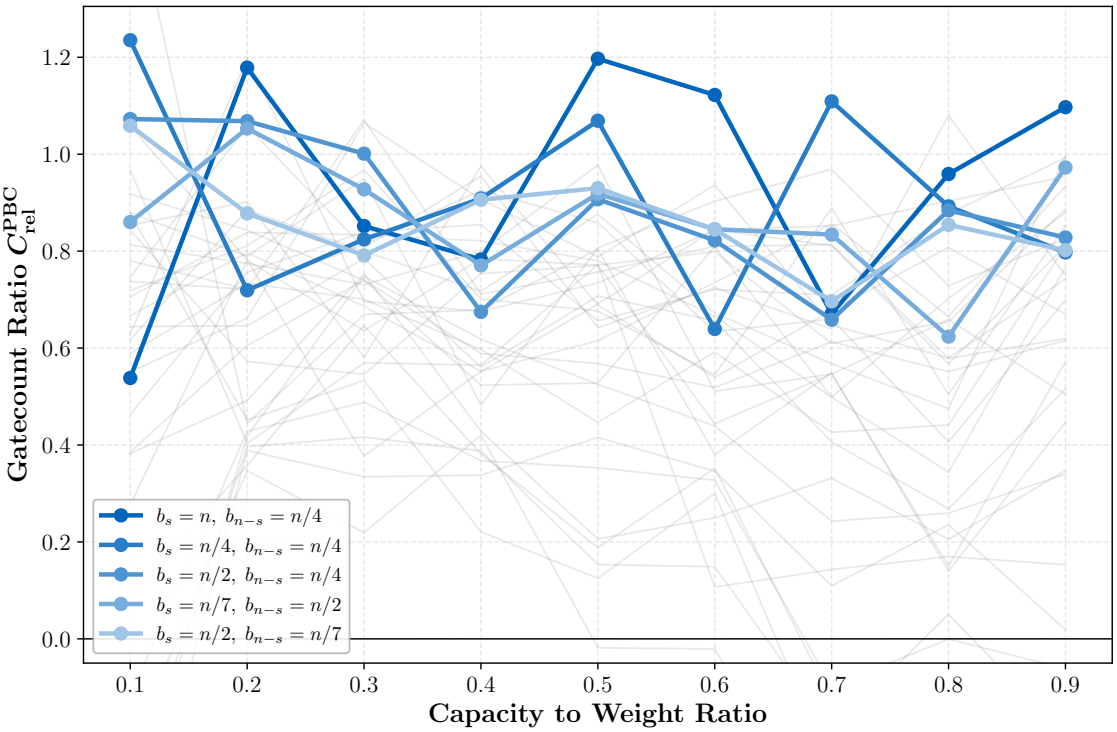


Figure 8: Bias sweep for the Profit-based Criterion (PBC) on weakly correlated knapsack instances. The x-axis is the capacity-to-weight ratio CWR, which groups instances by capacity-tightness, and the y-axis reports the mean gate count ratio (from Eq. (91)) relative to the Baseline Approach (BA) for each bias pair. Colored curves mark the five best bias pairs. The best average performance is obtained at $b_s^{\text{PBC}} = n$ and $b_{n-s}^{\text{PBC}} = \frac{n}{4}$.

Figure 7 shows that for the PBC criterion on uncorrelated instances the best bias combination on average is $b_s^{\text{PBC}} = n$ and $b_{n-s}^{\text{PBC}} = \frac{n}{4}$. Figure 8 shows that for the PBC criterion on weakly correlated instances the best bias combination on average is also $b_s^{\text{PBC}} = n$ and $b_{n-s}^{\text{PBC}} = \frac{n}{4}$. While the next best performing combinations in both examples differ slightly, we observe two trends. First, the value $b_{n-s}^{\text{PBC}} = \frac{n}{4}$ is a good choice across the leading combinations of both instance classes. Second, the leading combinations within each class perform very similarly to each other on average, with deviations of at most 12% in the gate count ratio.

Reading both experiments together, and aiming for a choice that does not depend on the instance class (as elaborated on in Section 5.1), we settle on the single bias combination $b_s^{\text{PBC}} = n$ and $b_{n-s}^{\text{PBC}} = \frac{n}{4}$ for both instance classes, which is the nominal optimum in both cases. The QTG on the first s variables thus receives a large bias, so that the partial

greedy prefix receives a large amount of the amplitude, while the QTG on the remaining $n - s$ variables receives a distinctly weaker bias.

Both figures also show that the NAA with the PBC can improve on the naive BA implementation. We stress that this advantage is observed under two simultaneous best-case conditions: the depth s is chosen per instance as the instance-wise optimum, and the bias pair is the one selected on the very same data. It therefore establishes that an advantage exists somewhere in the parameter space, not that it is attainable at runtime. Whether it survives once the depth has to be selected without oracle knowledge is the subject of Section 5.4 and Section 5.5.

5.2.2 Bias Sweep for the Profit-and-Capacity-based Criterion (PCBC)

We now examine the optimal bias parameters for the PCBC. Figure 9 shows the result for uncorrelated instances, while Figure 10 shows the result for weakly correlated instances.

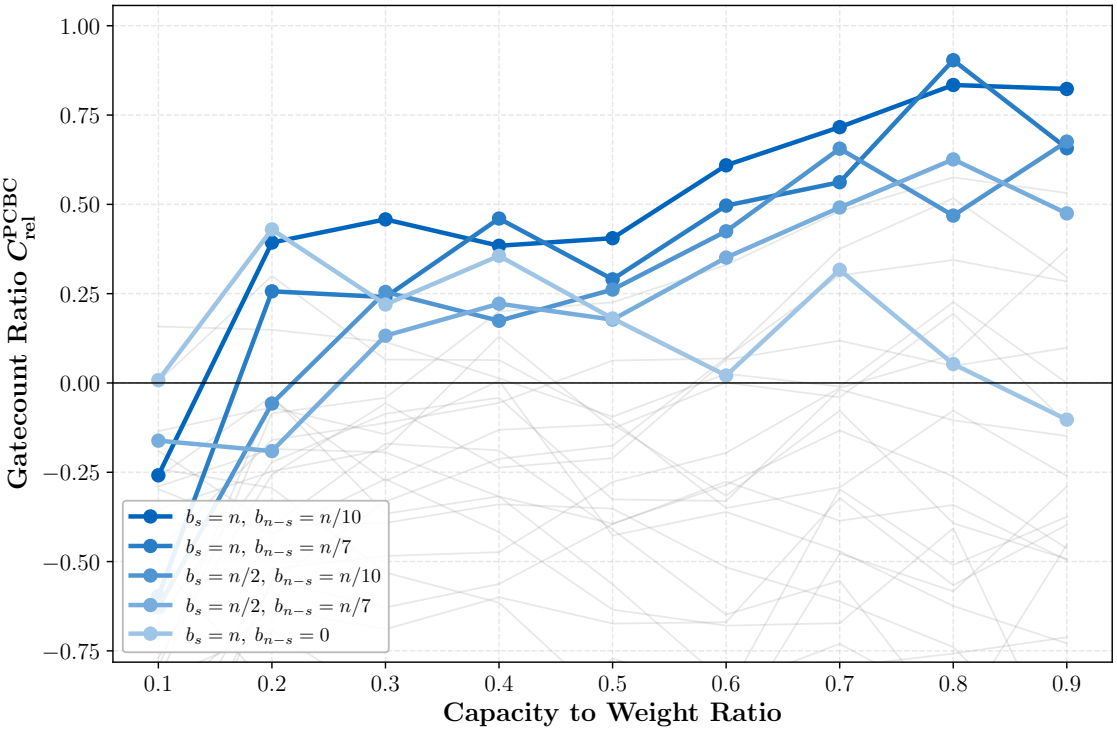


Figure 9: Bias sweep for the Profit-and-Capacity-based Criterion (PCBC) on uncorrelated knapsack instances. The x-axis is the capacity-to-weight ratio CWR, which groups instances by capacity-tightness, and the y-axis reports the mean gate count ratio (from Eq. (91)) relative to the Baseline Approach (BA) for each bias pair. Colored curves mark the five best bias pairs. The best average performance is obtained at $b_s^{\text{PCBC}} = n$ and $b_{n-s}^{\text{PCBC}} = \frac{n}{10}$.

For Figure 9, the best bias combination on average is $b_s^{\mathrm{PCBC}} = n$ and $b_{n-s}^{\mathrm{PCBC}} = \frac{n}{10}$, across the entire CWR range. Similarly, for Figure 10 the best bias combination on average is $b_s^{\mathrm{PCBC}} = n$ and $b_{n-s}^{\mathrm{PCBC}} = 0$. Different from the results in Section 5.2.1, the leading combination here stands out more clearly for the uncorrelated case, with the top 5 combinations deviating up to 25% in gate cost. For the weakly correlated case, the leading combination shows an advantage less clearly, with deviations of up to 16%.

Unlike the PBC, where both instance classes share the same optimum, the PCBC optima differ more substantially. We therefore adopt class-dependent biases for subsequent experiments: $b_s^{\mathrm{PCBC}} = n$ and $b_{n-s}^{\mathrm{PCBC}} = \frac{n}{10}$ for uncorrelated instances, and $b_s^{\mathrm{PCBC}} = n$ and $b_{n-s}^{\mathrm{PCBC}} = 0$ for weakly correlated instances. As argued in Section 5.1, this remains admissible because the correlation class can be estimated classicaly.

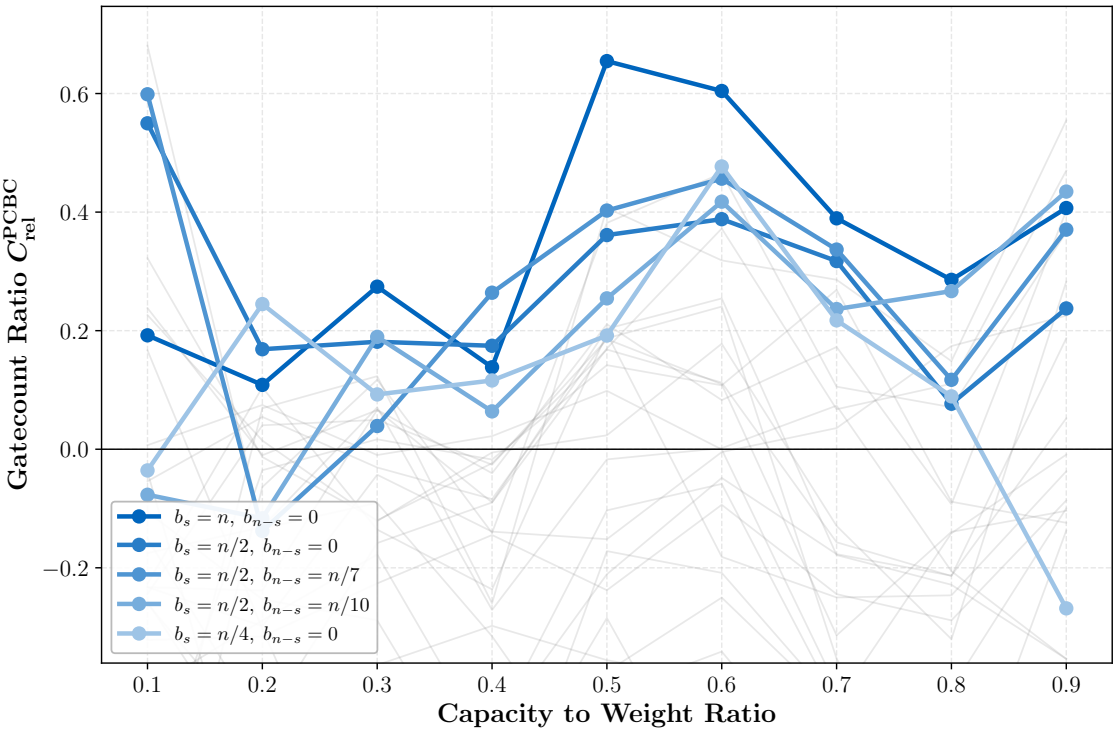


Figure 10: Bias sweep for the Profit-and-Capacity-based Criterion (PCBC) on weakly correlated knapsack instances. The x-axis is the capacity-to-weight ratio CWR, which groups instances by capacity-tightness, and the y-axis reports the mean gate count ratio (from Eq. (91)) relative to the Baseline Approach (BA) for each bias pair. Colored curves mark the five best bias pairs. The best average performance is obtained at $b_s^{\mathrm{PCBC}} = n$ and $b_{n-s}^{\mathrm{PCBC}} = 0$.

These results agree directionally with arguments made in Section 5.1. We expect the average Hamming distance to the nearest improving solution to be larger for weakly correlated instances than for uncorrelated instances, because the greedy solution is less likely to include items that are almost certainly in the optimal solution. Recalling $b_{\text{opt}} = \frac{n}{\Delta} - 2$ from Section 2.6.4, the choice $b_{n-s}^{\text{PCBC}} = 0$ corresponds to $\Delta \approx \frac{n}{2}$, i.e. a reference solution that carries essentially no information about the remaining $n - s$ variables, whereas $b_{n-s}^{\text{PCBC}} = \frac{n}{10}$ corresponds to a considerably smaller Δ. The ordering of the two classes therefore matches the ordering observed in Figure 6. The absolute values do not: both sweeps favour a distinctly weaker bias than the measured Δ of Figure 6 would suggest. This is expected, since b_{n-s}^{q} acts on a state that has already passed through the partial state preparation, the IIF and the partial oracle, so the effective distance it has to bridge is not the greedy-to-optimum distance measured in Section 5.1.

Just as in Section 5.2.1, these results indicate a potential improvement of the NAA with the PCBC over the naive BA implementation, again only under the combination of instance-wise optimal depth and in-sample optimal bias. We investigate this further in Section 5.5.

5.3 Inner Iterations

After having settled on bias values for both instance classes and partial amplification criteria, we can now evaluate the NAA. Before moving, however, to understanding how to pick promising depths (on average) in Section 5.4, we use the simulation across all depths (as in Section 5.2) to understand better what the IIF typically settles on as the number of inner iterations, r_{in}^{*}. If the IIF does in fact amplify states in a promising way and does not incur too high a gate count doing it, we would expect r_{in}^{*} to be different from 0 on average.

In this setup, just as in Section 5.2, we sweep the depth s across the entire range of $[1, n - 1]$ and try to improve upon the greedy solution only once (again as a computationally tractable proxy to a full scale optimisation). As depicted in Figure 5, we store all the parameters of the simulation and proceed with an evaluation based on the results. For bias parameters, we use the values identified in Section 5.2.

All the figures in Section 5.3.1 and Section 5.3.2 portray the same things: For each instance, we pick the best (w.r.t. Eq. (91)) depth s at which r_{in}^{*} is 0 (blue) and non-zero

(orange). We then plot the distribution of the gate count ratio for both cases in order to extract information about the cost a non-zero r_{in}^* incurs.

5.3.1 Inner Iteration Behavior for the Profit-based Criterion (PBC)

For the PBC, Figure 11 shows a striking result. Runs at which r_{in}^* is non-zero tend to be much more costly for both the uncorrelated and the weakly correlated instance class.

In particular, for uncorrelated instances, selecting the best depth subject to $r_{\text{in}}^* > 0$ yields an average gate count ratio of -2.5, while selecting the best depth subject to $r_{\text{in}}^* = 0$ yields an average gate count ratio of 1.2. Since the gate count ratio of (91) is a base-2 logarithm, this corresponds to the NAA being roughly 5.7 times more expensive than the BA in the former case and roughly 2.3 times cheaper in the latter. Strikingly, when counting each instance once and selecting the depth s with the largest observed gate count ratio, the corresponding best-depth run has $r_{\text{in}}^* = 0$ for 97.6% of the uncorrelated instances. A similar picture emerges for the PBC on weakly correlated instances, where the average gate count ratio for non-zero r_{in}^* is -3.7, while the average gate count ratio for $r_{\text{in}}^* = 0$ is 1.0. Similarly, when counting each instance once and selecting the depth s with the largest observed gate count ratio, the corresponding best-depth run has $r_{\text{in}}^* = 0$ for 98.5% of the weakly correlated instances.

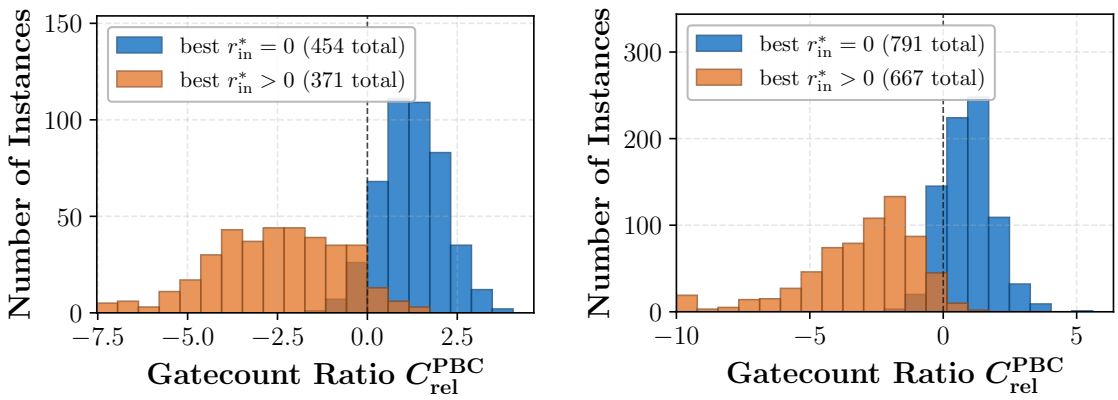


Figure 11: Gate count ratio distributions for the Profit-based Criterion (PBC), conditioned on whether the best-depth run has $r_{\text{in}}^* = 0$ (blue) or $r_{\text{in}}^* > 0$ (orange), for uncorrelated instances (left) and weakly correlated instances (right). The x-axis is the gate count ratio relative to the Baseline Approach (BA) and the y-axis is the frequency of instances in the distribution. Runs with $r_{\text{in}}^* > 0$ are shifted toward substantially worse costs in both instance classes.

These results suggest that the IIF preferably returns $r^*_{\text{in}} = 0$ and the partial states do not actually get amplified. Moreover, it suggests that forcing an amplification (orange distributions in Figure 11) is very expensive in gate count, at a limited benefit. However, we do also recognize the fact that, at optimal depth, $r^*_{\text{in}} = 0$ yields an advantage over the BA on average for both instance classes. We discuss the results, together with the PCBC in Section 5.3.3.

5.3.2 Inner Iteration Behavior for the Profit-and-Capacity-based Criterion (PCBC)

The results for the PCBC in Figure 12 are similar but more pronounced compared to the PBC.

In particular, for uncorrelated instances, the average gate count ratio is -3.9 when the best depth subject to $r^*_{\text{in}} > 0$ is selected, compared with 0.3 when the best depth subject to $r^*_{\text{in}} = 0$ is selected. Moreover, when each instance is counted once and the depth s with the largest observed gate count ratio is selected without restricting r^*_{in}, the corresponding run has $r^*_{\text{in}} = 0$ for 99.1% of instances. The weakly correlated instances exhibit the same pattern: the two conditional average gate count ratios are -3.5 and 0.8, respectively, while 98.7% of the unrestricted best-depth runs have $r^*_{\text{in}} = 0$.

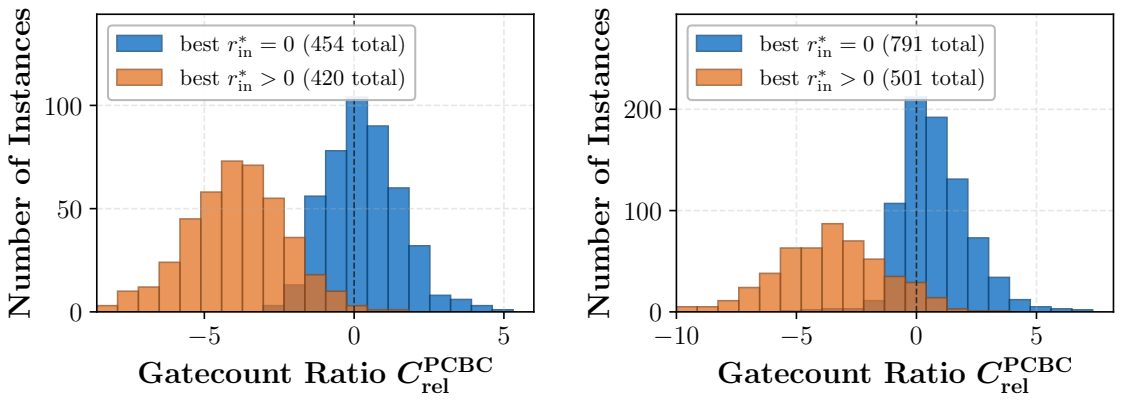


Figure 12: Gate count ratio distributions for the Profit-and-Capacity-based Criterion (PCBC), conditioned on whether the best-depth run has $r^*_{\text{in}} = 0$ (blue) or $r^*_{\text{in}} > 0$ (orange), for uncorrelated instances (left) and weakly correlated instances (right). The x-axis is the gate count ratio relative to the Baseline Approach (BA) and the y-axis is the frequency of instances in the distribution. Runs with $r^*_{\text{in}} > 0$ are shifted toward substantially worse costs in both instance classes.

Compared to the PBC, the PCBC shows an even bigger tendency to settle on $r_{\text{in}}^* = 0$ and the cost of non-zero r_{in}^* is even more pronounced. One would of course expect the blue distributions to match across the same instance class, but the difference can be explained by the different biases chosen in Section 5.2 and the different item ordering between the PBC and the PCBC. We discuss the results, together with the PBC in Section 5.3.3.

5.3.3 Interpretation of the Results

The results are discouraging. It appears that given the bias in Section 5.2, it is advantageous for the IIF to not amplify partially promising states. Rather, the IIF seems to preferably degenerate into a state preparation routine, where the QTG employs a different bias on the first s variables and the remaining $n - s$ variables. There are several reasons for this. First, the biases chosen (those which showed a possible advantage over BA) heavily concentrate amplitude into the greedy prefix. Since the greedy partial state is a partially marked state at almost all depths[9], the initial amplitude of the marked subspace is already so large that no amplification is performed. Second, in the cases where amplification could be performed, the gate counts of the IIF (PBC and PCBC) are too high relative to their savings in the outer stage to be beneficial. Third, the IIF performs amplification preferably when the depth s is large (since the criteria become tighter as s approaches n), but in this regime the gate cost scales with s and outweighs any benefit to the starting amplitudes for the outer stage. Nevertheless, the NAA with $r_{\text{in}}^* = 0$ can still improve upon the BA in some cases, which motivates further investigation in the following sections.

5.4 Runtime Depth Prediction

So far, this chapter has established a theoretical best-case advantage of the NAA over the BA in Section 5.2, while arguing that the IIF is effective only to a very limited degree in Section 5.3. All results presented so far have been based on a simulation where the depth s at which the IIF is applied was swept and the instance-wise optimum taken. Of course, in order to be useful, depth needs to be predicted at runtime from efficiently computable metrics. In this section we attempt to identify such depths.

Using the biases from Section 5.2, we again sweep the depth fraction $\frac{s}{n}$ across the entire range of the problem. For computational reasons, we again limit the simulation to only improve the greedy solution once (this condition is relaxed in Section 5.5).

[9] A partial greedy solution is almost always marked, because improving item swaps can almost certainly be found in the remaining $n - s$ variables.

5.4.1 Depth Fraction Sweep

Figure 13 and Figure 14 show the results for both amplification criteria and both correlation types. Strikingly, we recognize a clear disadvantage for the NAA across all instances and depths on average. We observe the PBC to perform best for low depths, a result that deteriorates quickly as depth and correlations increase. The PCBC performs worse for uncorrelated instances and shows an optimum for both correlation types, albeit in the negative gate count regime.

An important sanity check here is the behaviour as s approaches 0. Since the PBC and the BA use the same item ordering, we would expect the performance to converge (since NAA on $s = 0$ is the BA up to some constant overhead), which can be observed. For the PCBC, which uses density ordered items, as s approaches 0, we essentially compare two different versions of the BA, one value-ordered and one density-ordered. Since the BA implements (39), we expect the density-ordered version to perform worse.

The 1σ envelopes in Figure 13 and Figure 14 indicate that there are in fact depths for which some instances show a better performance for the NAA than for the BA, a statement that does not hold on average across all instances.

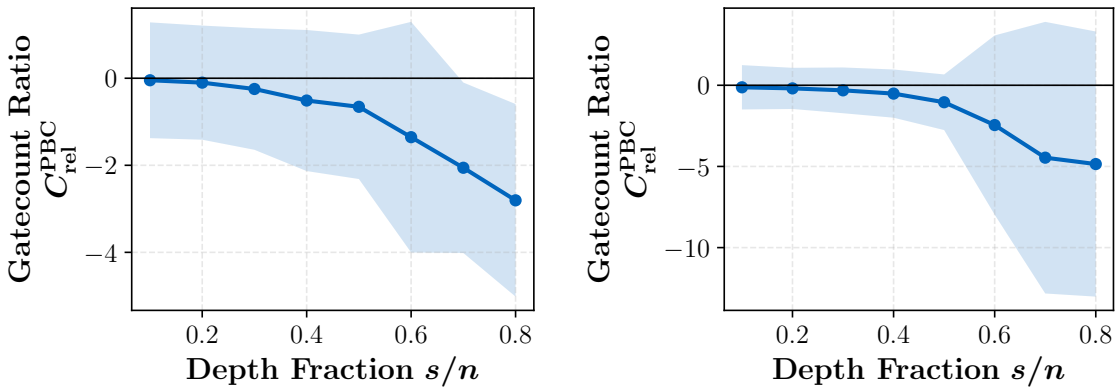


Figure 13: Gate count ratio across depth fractions for the Profit-based Criterion (PBC) on uncorrelated (left) and weakly correlated (right) instances. The x-axis is the depth fraction $\frac{s}{n}$ and the y-axis is the mean gate count ratio relative to the Baseline Approach (BA). Shaded bands show the 1σ envelope. Average performance is best at low depths and deteriorates as depth and correlation increase.

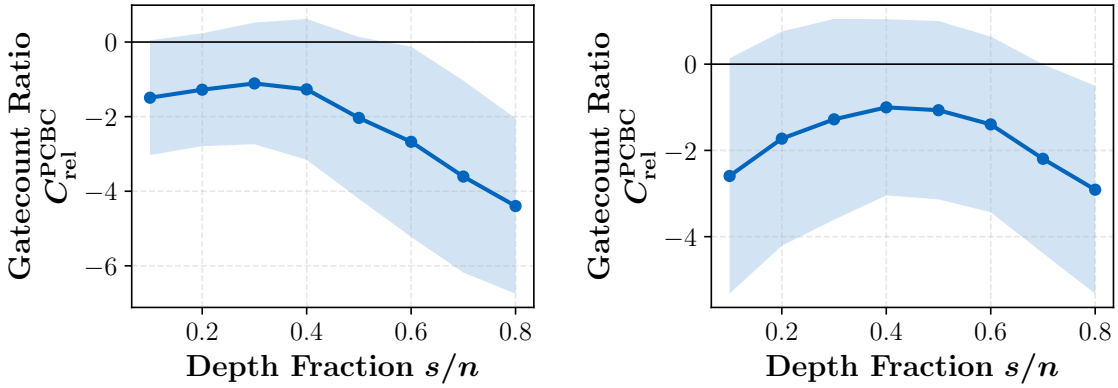


Figure 14: Gate count ratio across depth fractions for the Profit-and-Capacity-based Criterion (PCBC) on uncorrelated (left) and weakly correlated (right) instances. Axes and bands are as in Figure 13. Both correlation types show an optimum at moderate depths, which nevertheless stays below the Baseline Approach (BA) on average.

The fact that finding depths that are advantageous for the NAA is difficult is further supported by Figure 15 and Figure 16. Plotting the distribution of the most advantageous depth fraction per instance, we observe that for the PBC, although skewed to low depths, significant fractions of instances have their best depth fraction at higher values. For the PCBC the distribution seems even broader and somewhat more peaked at moderate depths, agreeing well with the results in Figure 14.

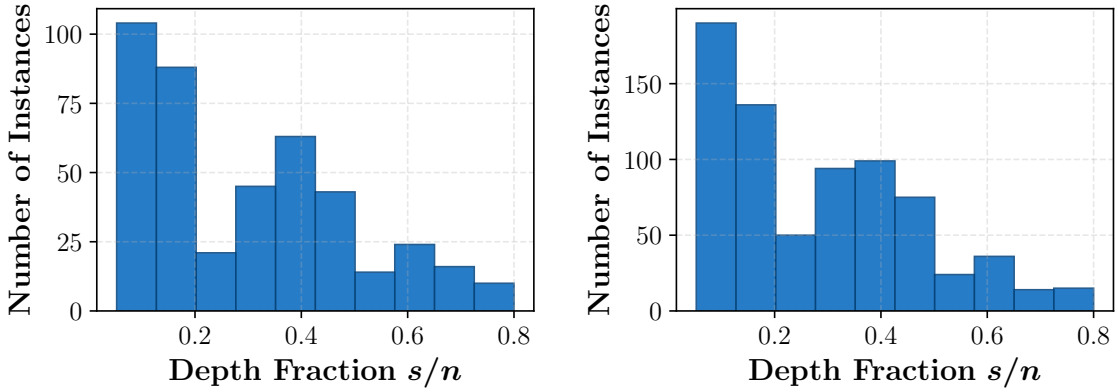


Figure 15: Distribution of instance-wise best depth fractions for the Profit-based Criterion (PBC) on uncorrelated (left) and weakly correlated (right) instances. The x-axis is the depth fraction $\frac{s}{n}$ at which each instance performs best and the y-axis is the number of instances for which the corresponding $\frac{s}{n}$ is optimal. The distribution is skewed towards low depths, but a substantial fraction of instances peaks elsewhere.

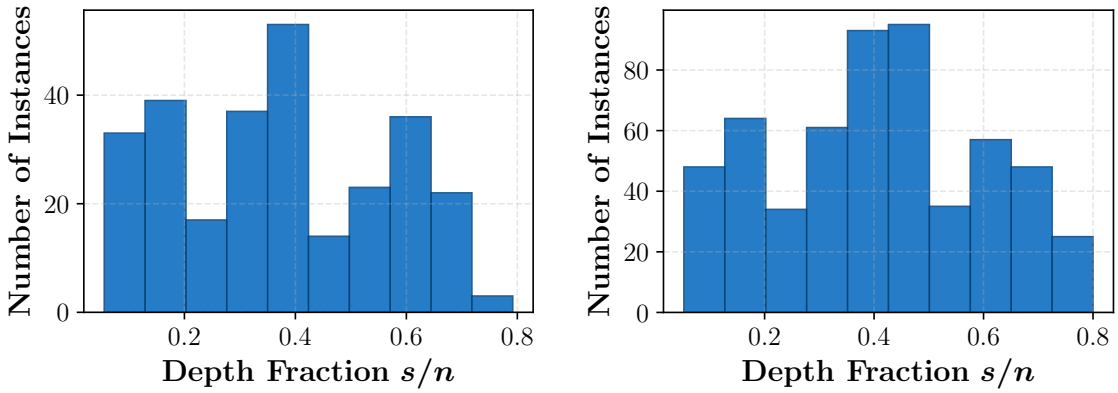


Figure 16: Distribution of instance-wise best depth fractions for the Profit-and-Capacity-based Criterion (PCBC) on uncorrelated (left) and weakly correlated (right) instances. Axes are as in Figure 15. The broad spread shows that favourable depths are not concentrated at a single easily predictable value.

5.4.2 Depth Schedule

One way of mitigating the effect of depth unpredictability, is to choose a set schedule at which to iterate the sweep of different depths. The idea behind this is to essentially cumulatively cover best depths by sweeping over depth fractions at runtime. Empirically, we choose a schedule of

$$\frac{s}{n} = 0.3, 0.4, 0.2, 0.5, 0.1, 0.6 \tag{95}$$

for the PBC and

$$\frac{s}{n} = 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0. \tag{96}$$

for the PCBC. Even though the choices are motivated by the results from Figure 15 and Figure 16, they are not necessarily optimal. Even if some optimal choice could be found here, e.g. by optimizing the schedule for the cumulative coverage of depths for the datasets at hand, the results would hardly generalize to other datasets.

Figure 17 and Figure 18 show the cumulative coverage achieved by these schedules, i.e. the fraction of instances whose best depth fraction has been visited after a given number of scheduled candidates. For the final benchmarking in Section 5.5, we adapt this schedule, in the hope of covering best depths early enough to allow for advantageous performance.

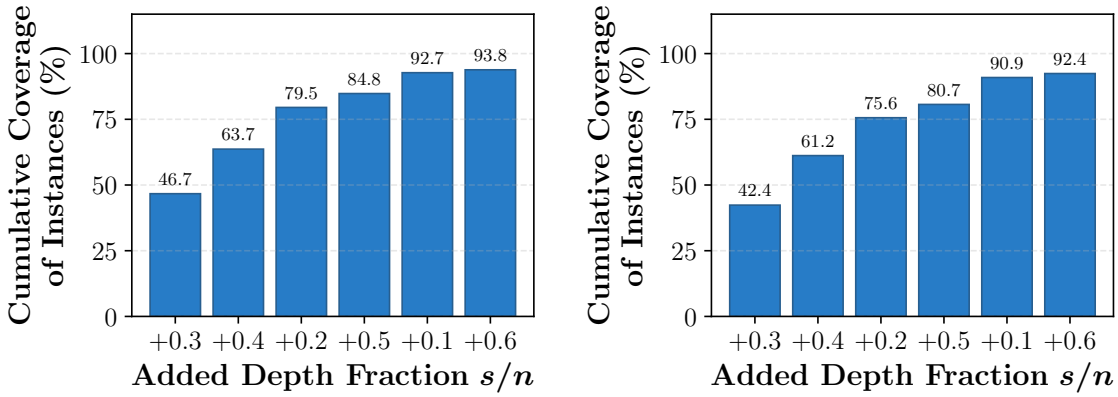


Figure 17: Cumulative candidate coverage of the depth schedule for the Profit-based Criterion (PBC) on uncorrelated (left) and weakly correlated (right) instances. The x-axis counts how many scheduled depth candidates have been tried and the y-axis is the cumulative fraction of instances whose best depth has already been visited.

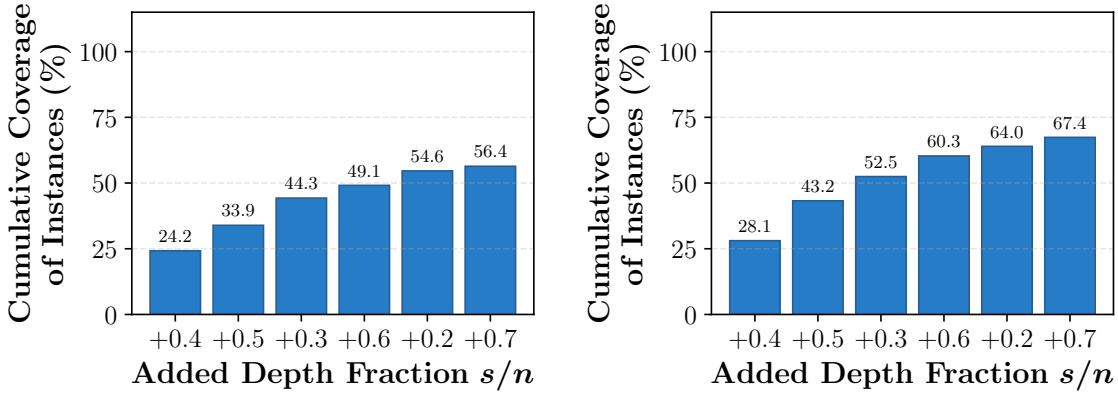


Figure 18: Cumulative candidate coverage of the depth schedule for the Profit-and-Capacity-based Criterion (PCBC) on uncorrelated (left) and weakly correlated (right) instances. Axes are as in Figure 17.

5.5 Fixed Budget Performance

Finally, this section examines the performance of the NAA under a fixed budget constraint, described in Section 3.6.2. Concretely, we show the optimality gap from (93) against the budget exponent δ, when the budget is of the form

$$\mathcal{B} = \kappa \cdot n^{\delta} \tag{97}$$

with $\kappa = 1000$ an empirical constant. This comparison is the fairest one between the two approaches: the biases are chosen as in Section 5.2 and the depth schedule as in Section 5.4.2 is used. The incumbent is improved starting from the greedy solution until

the budget termination condition is met. Performing better in this regard would mean reaching a higher optimality gap at a smaller budget.

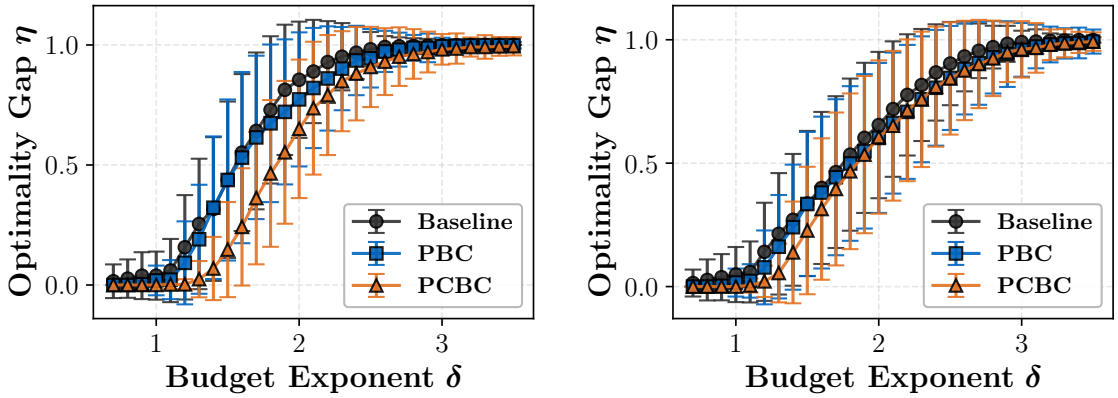


Figure 19: Fixed-budget performance on uncorrelated (left) and weakly correlated (right) knapsack instances. The x-axis is the budget exponent δ in (97) and the y-axis is the optimality gap from (93), where larger values indicate better solution quality. Curves show the mean over all instances with $\pm 1\sigma$ error bars. The Profit-based Criterion (PBC) stays close to the Baseline Approach (BA), while the Profit-and-Capacity-based Criterion (PCBC) does not show an advantage.

Figure 19 shows the result on both instance classes. For uncorrelated instances, the PBC tracks the BA closely in performance. Differences are visible only for solution qualities that approach the optimum. The PCBC however, performs much more poorly compared to the BA. The quality of solution is worse throughout the budget sweep and only converges as the budget becomes large enough for all approaches to solve the problem to optimality.

For weakly correlated instances, the PBC again tracks the performance of the BA closely. However, the PCBC performs much better on weakly correlated instances than it does on uncorrelated instances. While still at a minimal disadvantage compared to the BA, the PCBC is able to reach a solution quality that is only slightly worse than the BA for a large range of budgets. For small budgets the gap is larger, as the PCBC does not yet have enough budget to complete an entire outer stage of the NAA.

From Figure 19 we can conclude that for the depth schedule from Section 5.4.2 and the bias choices from Section 5.2, the NAA approach does not show merit compared to the BA. The reasons for this are manifold and we discuss them in some detail here.

Firstly, we should recognize that for the problem instances considered in this thesis, the greedy solution is already of high quality. As shown in Figure 6, the optimum and the greedy solution share an overwhelming number of bits. This in turn means, that any improvement in algorithmic design between the NAA and the BA operates on a very small margin of improvement. Secondly, as shown in Section 5.3, the IIF oftentimes settles on $r^*_{\text{in}} = 0$, which means that the NAA does not actually perform any amplification of partially promising states. This is a direct consequence of the ideal bias being large and hence a lot of amplitude being received by the greedy partial state. Concentrating that much amplitude in the greedy prefix can in fact have negative effects, since the greedy state on the whole subspace cannot improve upon any incumbent. The IIF hence prefers to degenerate into a stepwise QTG, where the bias changes between the two state preparations. Thirdly, we have shown in Section 5.3, that in cases in which amplification of partial states does occur, the benefit from warm-starting amplitudes to the outer stage is more than outweighed by the additional gate cost of the PBC and the PCBC. In particular, we can see that in Figure 19 the gate cost added by the PCBC very quickly outweighs any benefit. Lastly, Section 5.4 portrays in some detail the fact that efficiently predicting the nesting depth s is hard. We opt for a schedule of depths that covers many of the best depths in the problem, but Figure 19 speaks to the fact that the gate cost incurred by querying at wrong depths significantly outweighs the benefit of partial amplification, most strikingly so for the PCBC.

In order to gauge the impact of the large amplitude distribution in the greedy prefix, we also investigate the behaviour of the algorithms in an unbiased setting in Section 5.6. This is motivated by the fact that the disadvantage of the NAA is minimal for the PBC across instance classes and the PCBC for weakly correlated instances.

5.6 Unbiased Fixed Budget Performance

Having thoroughly studied the performance of the NAA, we have identified reasons for its lack of performance compared to the BA. One such reason is the large amplitude clustering on the partial greedy state, which in many cases prohibits the IIF from performing partial amplification. Finally, in this section we set out to investigate the regime where neither the BA nor the NAA operate with any bias in the QTG. This regime seems promising to find some advantage of the NAA, since the partial marked states receive much less amplitude to begin with. In order to properly quantify any possible advantage, we first

set out to determine whether certain capacity-to-weight ratio regimes are promising in the unbiased case, to then perform the fixed budget performance comparison, very similar to that of Section 5.5.

Following the same pipeline as in Section 5.2, but setting all biases to zero:

$$b = b_s^q = b_{n-s}^q = 0, \tag{98}$$

we produce Figure 20. Again, all depths are swept and the instance-wise optimum is chosen. The figure clearly indicates two different regimes for improving the greedy solution once: the PBC performs well on uncorrelated instances with a capacity-to-weight ratio of CWR > 0.6, while the PCBC performs well on weakly correlated instances with a capacity-to-weight ratio of CWR < 0.5. In these two regimes, the respective criterion very clearly separates itself from the BA, which is why choosing these regimes for the fixed budget performance comparison is promising. As in Section 5.2, we do note that a best-depth sweep which only improves the greedy solution once is a best-case scenario which should be understood as a proxy to investigate computationally untractable regimes. We further note that both regimes are selected on the same instances on which they are subsequently evaluated. Regardless, the results can be verified by the fixed budget performance comparison, which is what we set out to do next.

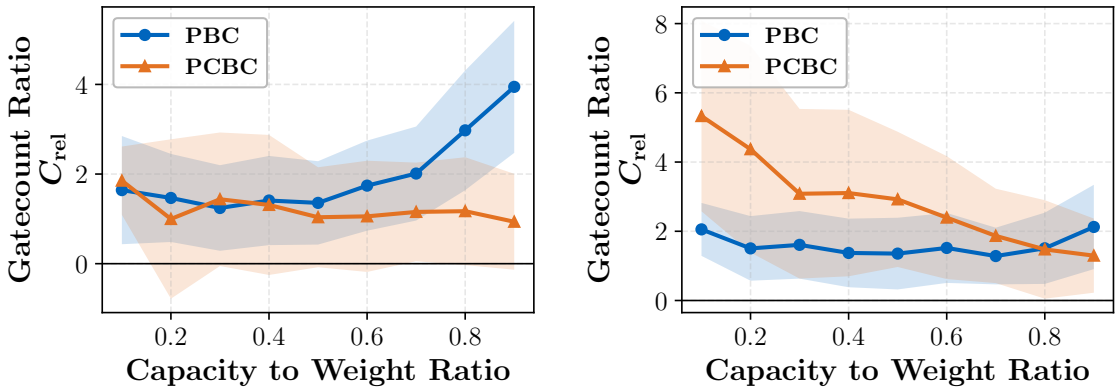


Figure 20: Capacity-to-weight ratio sweep for the unbiased Nested Amplitude Amplification (NAA) on uncorrelated (left) and weakly correlated (right) knapsack instances. The x-axis is the capacity-to-weight ratio CWR and the y-axis reports the mean gate count ratio relative to the Baseline Approach (BA) after choosing the best depth per instance. Shaded bands show the 1σ envelope. The promising regimes are CWR > 0.6 for the Profit-based Criterion (PBC) on uncorrelated instances and CWR < 0.5 for the Profit-and-Capacity-based Criterion (PCBC) on weakly correlated instances.

Simulating Section 5.5 again without bias requires some thought on the allocated budget. Since a well chosen bias speeds up the optimization substantially [20], the constant κ in the budget term (97) must be increased to allow the plot to reach the regime where all approaches reach optimality. Empirically, we choose $\kappa = 2500$. We then simulate the two regimes identified in Figure 20. Apart from the restriction on the CWR and the change in bias, the simulation follows Section 5.5. Choosing the closed-gap level at which the average improvement between the BA and the respective criterion is largest, we compute the corresponding improvement in the termination-cost exponent, $\Delta\delta^q = \delta_{\text{baseline}} - \delta^q$, for each instance. Plotting the mean of these datapoints in instance-size bins yields Figure 22. For both criteria, although much more pronounced for PCBC, the advantage increases with problem size. This is a promising result, since it indicates that the NAA can outperform the BA in the regimes studied in this section, particularly for larger instance sizes.

Overall, Figure 21 and Figure 22 show two things: the PBC performs well on uncorrelated instances and the PCBC performs well on weakly correlated instances in the unbiased case (although a similar trend has already be observed in Figure 19). To understand the PBC performance, we recall that the QTG only prepares capacity-feasible states, so every partial state satisfies $\sum_{m=1}^{s} w_m x_m \leq c$ by construction. On weakly correlated instances

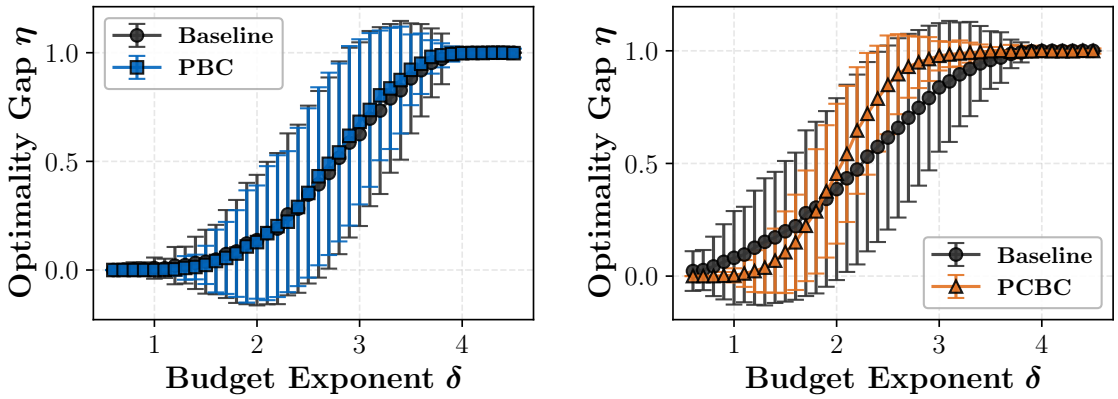


Figure 21: Unbiased fixed-budget performance for the Profit-based Criterion (PBC) on uncorrelated instances with CWR > 0.6 (left) and the Profit-and-Capacity-based Criterion (PCBC) on weakly correlated instances with CWR < 0.5 (right). The x-axis is the budget exponent δ from (97) and the y-axis is the optimality gap, where larger values mean a higher solution quality. Curves show the mean over all instances with $\pm 1\sigma$ error bars. In these selected regimes both criteria outperform the Baseline Approach (BA), with a clearly stronger gain for the Profit-and-Capacity-based Criterion (PCBC).

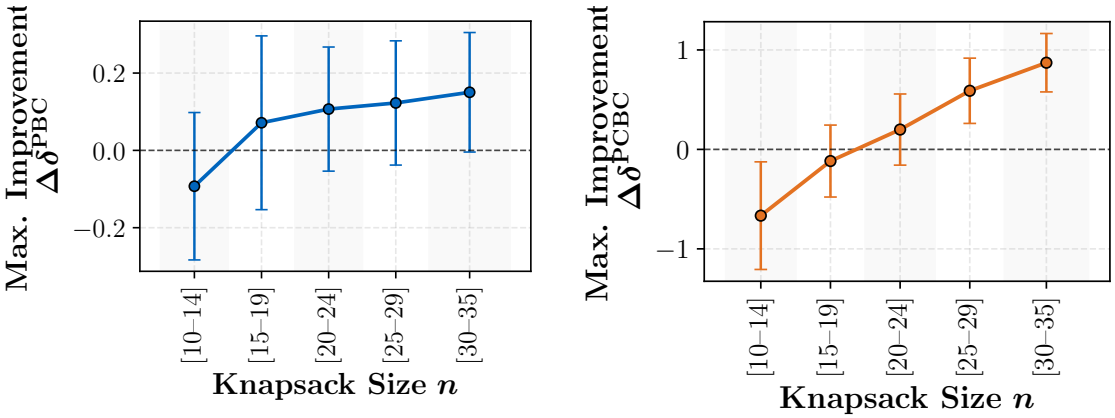


Figure 22: Unbiased size scaling for the Profit-based Criterion (PBC) on uncorrelated instances with CWR > 0.6 (left) and the Profit-and-Capacity-based Criterion (PCBC) on weakly correlated instances with CWR < 0.5 (right). The x-axis is the instance size n (binned) and the y-axis is the improvement in termination-cost exponent $\Delta\delta^q$, where larger values mean a larger budget advantage over the Baseline Approach (BA), at the same quality of solution. 1σ errorbars in a bin are shown. The Profit-based Criterion (PBC) improves only modestly with size, whereas the Profit-and-Capacity-based Criterion (PCBC) shows the stronger scaling trend.

the instance generator of Section 4.1 [9] sets $p_m \approx w_m$, so substituting this into (55) yields the following inequality for partial states that receive an amplitude:

$$y^{\mathrm{inc}} - \sum_{m=s+1}^{n} p_m < \sum_{m=1}^{s} w_m x_m \leq c. \tag{99}$$

With $p_m \approx w_m$ the profit of every feasible solution roughly equals the capacity it consumes, so neither a feasible solution nor the incumbent can carry a profit that exceeds c on average. Therefore the two bounds in (99) lie at least $\sum_{m=s+1}^{n} p_m$ apart. On weakly correlated instances the PBC consequently constrains the same quantity the QTG already constrains and only partial states that leave a large part of the capacity unused get rejected. On uncorrelated instances the PBC cannot be reduced to an inequality of partial weights and is hence comparatively tighter. Analogously, the observation for the PCBC can be explained by looking at the criterion in detail. As can be seen in (59), the number of states that receive any amplitude decreases as ρ_{m^*} decreases (since every partial state that is not excluded by the QTG has a non-negative remaining capacity). The crucial observation then follows from the behaviour of ρ_{m^*}. Uncorrelated instances' profits and weights are sampled independently, so the maximum is attained by an item of small weight and large profit and has the potential of maximally reaching up to $p_{\max}/w_{\min}$. On

weakly correlated instances $p_m \leq w_m + r/10$ bounds every efficiency by $1 + r/(10w_m)$, which is not dominated by outliers as much as the uncorrelated case is. It hence follows that the PCBC is comparatively tighter on weakly correlated instances, which explains the performance difference between the two instance classes.

In conclusion, if the NAA is indeed allowed to perform amplification on partial states, because the marked greedy partial bitstring does not receive a large amount of the amplitude (by means of low bias), advantageous regimes for the NAA can be identified. Of course this result is of theoretical interest, since a fair benchmarking of the NAA needs to happen against the BA at its optimal bias, as shown in Section 5.5. However, the results underpin the analysis made in Section 5.5 and open up the possibility of finding advantageous regimes for problem instances where the greedy solution is much worse in quality.

6 Conclusion

This thesis introduces the NAA, a nested AA hot-starting approach to the binary knapsack problem. To answer the question posed in Chapter 1, we compared the performance of the NAA to the BA, measuring solution quality by the optimality gap of (93) at a fixed gate count budget. On a high level, we found that in a biased setting, the NAA tends to degenerate into a depth-dependent QTG hot-start which does not actually perform any amplification of the partial subspace. Consequently, the NAA does not outperform the BA in any investigated regime in the biased case for both amplification criteria we studied. In contrast, the unbiased setting produces certain instance regimes and parameter choices in which amplification of the partial subspace is useful and can, under the conditions detailed in Section 5.6 (conditions w.r.t. depth selection, CWR and sufficient budget), produce a visible advantage (in terms of solution quality to gate count) over the BA.

Independently of this empirical outcome, the thesis provides a complete design and in-depth analysis of the nested approach as a hot-start to GAS. Chapter 3 proposes the state preparation operators of the inner and outer stage, derives and specifies the PBC and the PCBC as amplification criteria that mark promising partial assignments, and provides an exact gate count analysis of the proposed algorithm. This analysis remains valid regardless of the empirical outcome and can be reused, at least in spirit, for any nested AA approach. Chapter 4 contributes the classical amplitude-tracking procedure and the metrics used to compare the two approaches. Parts of this work have been published at IEEE Quantum Week 2026 [23].

Chapter 5 identifies four main reasons for the lack of performance of the NAA in the biased case. Firstly, the strong optimal biases from Section 5.2 move a large amount of amplitude to the greedy prefix (which is almost always in the marked partial subspace), causing the IIF to perform almost no amplification. Secondly, if amplification is performed in the IIF (see Section 5.3), the benefits of the amplification are outweighed by the gate cost of the IIF for both the PBC and the PCBC. Thirdly, Section 5.1 shows that for the knapsack problem, the greedy solution is already of high quality, leaving little room for the NAA to improve upon the BA. Finally, Section 5.4 argues in detail that efficiently computing a depth s at which the NAA could show an advantage over the BA is not feasible. The depth schedule we propose to remedy this issue has been shown to be insufficient to tackle this task.

In the unbiased case investigated in Section 5.6, we are indeed able to find an advantage over the BA for certain regimes. Here, it appears to be beneficial to actually perform AA in the IIF. Since less amplitude is concentrated on the greedy prefix, the IIF contributes meaningfully to the amplification of the partial subspace and results in a visible advantage over the BA. As shown in Section 5.6, the PBC in this setting performs better on uncorrelated instances, while the PCBC performs better on weakly correlated instances. This advantage must, however, be interpreted with the necessary caution. A bias-free BA is known to be far from its optimum [20], so the comparison is not made against the strongest available baseline. The advantage additionally appears only within certain capacity-to-weight ranges, further limiting the generalisability of the result. In summary, the lack of bias enables the IIF to perform amplification, which leads to a visible advantage in the regimes identified.

The results of this thesis must be viewed in the context of decades of work on classical algorithms for the knapsack problem. The regime studied here does not pose a challenge to modern solvers, which are typically based on dynamic programming, BnB, or both. High-performance algorithms such as Combo [66] solve knapsack instances to proven optimality, often in milliseconds, and handle instances with several thousand items in the same regime [67]. Even so, considerable effort has gone into constructing instances that remain hard in this classical setting. Pisinger [9] shows that for strongly correlated instances, the range r from which the weights are sampled markedly affects solver performance, with solution times for instances of a few hundred variables reaching seconds. Notably, the same study finds uncorrelated and weakly correlated instances, the two types used in our benchmarks, to be almost unaffected by an increase of the data range. At large ranges r, Minknap and Combo instead "frequently run out of space before running out of time" [9]. Jooken et al. [67] propose further instance types that are hard to solve classically, with optimal solutions not obtained within a time limit of two hours. Following Wilkening et al. [20], we note that the QTG stores the capacity in $O(\log c)$ qubits, whereas dynamic programming requires memory linear in c, which suggests instances with large capacities as the natural setting for a comparison against classical solvers. Overall, the instance sizes and types we benchmark on, constrained by the classical amplitude tracking of Section 4.2, do not challenge state-of-the-art classical solvers. Regardless, this thesis provided methodological insight into solving knapsack problems on a quantum computer.

Multiple limitations bound the scope of the conclusions we can draw. All results are obtained by classical amplitude tracking, imitating noiseless hardware, rather than by execution on real quantum hardware with its associated errors. Gate count serves as a proxy for runtime and ignores hardware-specific connectivity, compilation and error correction overheads. We investigated uncorrelated and weakly correlated instances, leaving other instance types to be evaluated.

In future work, investigating the fact that a QTG with depth dependent bias (in our case: when the IIF performs no amplification) can still be a better choice than a constant bias could be fruitful. Of course, theoretically there is an optimal set of biases for each choice, that pushes amplitude towards the optimal solution to arbitrary accuracy. Naturally designing such a bias schedule would require knowledge of the optimal solution, which is not available in practice. Still, designing problem- (or even instance-) aware bias schedules promises to be effective in further improving the performance of the BA. Additionally, other criteria than the PBC and the PCBC could be investigated. In particular, the PCBC can be understood as a zero-order criterion of a family of polynomial criteria: it fills the knapsack's remaining capacity with the remaining items of the best efficiency. It is however possible to construct a higher order polynomial criterion by finding a function constrained by the item with the best remaining efficiency and the worst remaining efficiency. The knapsack would then be filled with the effective remaining items of an efficiency described by this polynomial. In general, this approach need not remain exact, i.e. the optimal solution might well be excluded by the derived profit bound. In the context of a poorly performing NAA in the biased case, however, heuristical approaches could be investigated as an alternative to the PBC and the PCBC. Finally, it would be interesting to investigate the NAA in a more general setting, e.g. for other combinatorial optimization problems.

Bibliography

[1] H. Ehm, T. Ponsignon, and T. Kaufmann, "The Global Supply Chain Is Our New Fab: Integration and Automation Challenges," in *2011 IEEE/SEMI Advanced Semiconductor Manufacturing Conference*, May 2011, pp. 1–6. doi: 10.1109/ASMC.2011.5898164.

[2] R. M. Karp, "Reducibility among Combinatorial Problems," *Complexity of Computer Computations: Proceedings of a Symposium on the Complexity of Computer Computations, Held March 20–22, 1972, at the IBM Thomas J. Watson Research Center, Yorktown Heights, New York, and Sponsored by the Office of Naval Research, Mathematics Program, IBM World Trade Corporation, and the IBM Research Mathematical Sciences Department*. Springer US, Boston, MA, pp. 85–103, 1972. doi: 10.1007/978-1-4684-2001-2_9.

[3] M. N. Hess, "Quantum Optimization Algorithms for Industrial Problems," 2026, doi: 10.15488/21770.

[4] V. C. B. Camargo, L. Mattiolli, and F. M. B. Toledo, "A Knapsack Problem as a Tool to Solve the Production Planning Problem in Small Foundries," *Computers & Operations Research*, vol. 39, no. 1, pp. 86–92, Jan. 2012, doi: 10.1016/j.cor.2010.10.023.

[5] C. Flechsig, J. Lohmer, R. Lasch, B. Zettler, G. Scheider, and D. Eberts, "Automated and Optimized Lot-To-Order Matching in 300 Mm Semiconductor Facilities," in *2021 32nd Annual SEMI Advanced Semiconductor Manufacturing Conference (ASMC)*, May 2021, pp. 1–6. doi: 10.1109/ASMC51741.2021.9435730.

[6] P. J. Kolesar, "A Branch and Bound Algorithm for the Knapsack Problem," *Management Science*, vol. 13, no. 9, pp. 723–735, May 1967, doi: 10.1287/mnsc.13.9.723.

[7] D. Pisinger, "Core Problems in Knapsack Algorithms," *Operations Research*, vol. 47, no. 4, pp. 570–575, Aug. 1999, doi: 10.1287/opre.47.4.570.

[8] H. Kellerer, U. Pferschy, and D. Pisinger, *Knapsack Problems*. Berlin, Heidelberg: Springer, 2004. doi: 10.1007/978-3-540-24777-7.

[9] D. Pisinger, "Where Are the Hard Knapsack Problems?," *Computers & Operations Research*, vol. 32, no. 9, pp. 2271–2284, Sep. 2005, doi: 10.1016/j.cor.2004.03.002.

[10] O. H. Ibarra and C. E. Kim, "Fast Approximation Algorithms for the Knapsack and Sum of Subset Problems," *J. ACM*, vol. 22, no. 4, pp. 463–468, Oct. 1975, doi: 10.1145/321906.321909.

[11] S. Martello and P. Toth, *Knapsack Problems: Algorithms and Computer Implementations*. USA: John Wiley & Sons, Inc., 1990.

[12] T. Kadowaki and H. Nishimori, "Quantum Annealing in the Transverse Ising Model," *Physical Review E*, vol. 58, no. 5, pp. 5355–5363, Nov. 1998, doi: 10.1103/PhysRevE.58.5355.

[13] R. Fakhimi and H. Validi, "Quantum Approximate Optimization Algorithm (QAOA)," *Encyclopedia of Optimization*. Springer, Cham, pp. 1–7, 2023. doi: 10.1007/978-3-030-54621-2_854-1.

[14] M. Cerezo *et al.*, "Variational Quantum Algorithms," *Nature Reviews Physics*, vol. 3, no. 9, pp. 625–644, Sep. 2021, doi: 10.1038/s42254-021-00348-9.

[15] A. Abbas *et al.*, "Challenges and Opportunities in Quantum Optimization," *Nature Reviews Physics*, vol. 6, no. 12, pp. 718–735, Dec. 2024, doi: 10.1038/s42254-024-00770-9.

[16] L. K. Grover, "A Fast Quantum Mechanical Algorithm for Database Search," in *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, in STOC '96. New York, NY, USA: Association for Computing Machinery, Jul. 1996, pp. 212–219. doi: 10.1145/237814.237866.

[17] C. Durr and P. Hoyer, "A Quantum Algorithm for Finding the Minimum," no. arXiv:quant-ph/9607014. arXiv, Jan. 1999. doi: 10.48550/arXiv.quant-ph/9607014.

[18] A. Gilliam, S. Woerner, and C. Gonciulea, "Grover Adaptive Search for Constrained Polynomial Binary Optimization," *Quantum*, vol. 5, p. 428, Apr. 2021, doi: 10.22331/q-2021-04-08-428.

[19] M. Hess *et al.*, "Grover Adaptive Search with Problem-Specific State Preparation," no. arXiv:2602.08418. arXiv, Feb. 2026. doi: 10.48550/arXiv.2602.08418.

[20] S. Wilkening, A.-I. Lefterovici, L. Binkowski, M. Perk, S. P. Fekete, and T. J. Osborne, "A Quantum Algorithm for Solving 0-1 Knapsack Problems," *npj Quantum Information*, vol. 11, no. 1, p. 146, Aug. 2025, doi: 10.1038/s41534-025-01097-8.

[21] A. Wichert, "Nested Grover's Algorithm for Tree Search," *Entropy*, vol. 28, no. 1, p. 24, Jan. 2026, doi: 10.3390/e28010024.

[22] N. J. Cerf, L. K. Grover, and C. P. Williams, "Nested Quantum Search and Structured Problems," *Physical Review A*, vol. 61, no. 3, p. 32303, Feb. 2000, doi: 10.1103/PhysRevA.61.032303.

[23] L. Demmler and M. Hess, "A Nested Amplitude Amplification Protocol for the Binary Knapsack Problem." Apr. 2026.

[24] G. R. Bitran and A. C. Hax, "Disaggregation and Resource Allocation Using Convex Knapsack Problems with Bounded Variables," *Management Science*, vol. 27, no. 4, pp. 431–441, Apr. 1981, doi: 10.1287/mnsc.27.4.431.

[25] D. Pisinger and P. Toth, "Knapsack Problems," *Handbook of Combinatorial Optimization: Volume1–3*. Springer US, Boston, MA, pp. 299–428, 1998. doi: 10.1007/978-1-4613-0303-9_5.

[26] A. Wigderson, "P, NP and Mathematics – a Computational Complexity Perspective," *Proceedings of the International Congress of Mathematicians Madrid, August 22–30, 2006*. European Mathematical Society - EMS - Publishing House GmbH, pp. 665–712, May 2007. doi: 10.4171/022-1.

[27] A. Lucas, "Ising Formulations of Many NP Problems," *Frontiers in Physics*, vol. 2, Feb. 2014, doi: 10.3389/fphy.2014.00005.

[28] A. Montanaro, "Quantum Speedup of Branch-and-Bound Algorithms," *Physical Review Research*, vol. 2, no. 1, p. 13056, Jan. 2020, doi: 10.1103/PhysRevResearch.2.013056.

[29] C. Roch, A. Impertro, T. Phan, T. Gabor, S. Feld, and C. Linnhoff-Popien, "Cross Entropy Hyperparameter Optimization for Constrained Problem Hamiltonians Applied to QAOA," no. arXiv:2003.05292. arXiv, Aug. 2020. doi: 10.48550/arXiv.2003.05292.

[30] P. W. Shor, "Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer." Aug. 1995. doi: 10.1137/S0097539795293172.

[31] D. J. Bernstein, "Grover vs. McEliece," in *Post-Quantum Cryptography*, N. Sendrier, Ed., Berlin, Heidelberg: Springer, 2010, pp. 73–80. doi: 10.1007/978-3-642-12929-2_6.

[32] D. Anguita, S. Ridella, F. Rivieccio, and R. Zunino, "Quantum Optimization for Training Support Vector Machines," *Neural Networks*, vol. 16, no. 5, pp. 763–770, Jun. 2003, doi: 10.1016/S0893-6080(03)00087-X.

[33] A. Montanaro, "Quantum Algorithms: An Overview," *npj Quantum Information*, vol. 2, no. 1, p. 15023, Jan. 2016, doi: 10.1038/npjqi.2015.23.

[34] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, 2010.

[35] D. E. Knuth, *The Art of Computer Programming: Sorting and Searching, Volume 3*. Addison-Wesley Professional, 1998.

[36] C. H. Bennett, E. Bernstein, G. Brassard, and U. Vazirani, "Strengths and Weaknesses of Quantum Computing," *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1510–1523, Oct. 1997, doi: 10.1137/S0097539796300933.

[37] C. Zalka, "Grover's Quantum Searching Algorithm Is Optimal," *Physical Review A*, vol. 60, no. 4, pp. 2746–2751, Oct. 1999, doi: 10.1103/PhysRevA.60.2746.

[38] G. Brassard, P. Hoyer, M. Mosca, and A. Tapp, "Quantum Amplitude Amplification and Estimation," vol. 305. pp. 53–74, 2002. doi: 10.1090/conm/305/05215.

[39] M. Boyer, G. Brassard, P. Hoeyer, and A. Tapp, "Tight Bounds on Quantum Searching," *Fortschritte der Physik*, vol. 46, no. 4–5, pp. 493–505, Jun. 1998, doi: 10.1002/(SICI)1521-3978(199806)46:4/5<493::AID-PROP493>3.0.CO;2-P.

[40] H. Ominato, T. Ohyama, and K. Yamaguchi, "Grover Adaptive Search With Fewer Queries," *IEEE Access*, vol. 12, pp. 74619–74632, 2024, doi: 10.1109/ACCESS.2024.3403200.

[41] T. G. Draper, "Addition on a Quantum Computer," no. arXiv:quant-ph/0008033. arXiv, Aug. 2000. doi: 10.48550/arXiv.quant-ph/0008033.

[42] P. Shor, "Algorithms for Quantum Computation: Discrete Logarithms and Factoring," in *Proceedings 35th Annual Symposium on Foundations of Computer Science*, Santa Fe, NM, USA: IEEE Comput. Soc. Press, 1994, pp. 124–134. doi: 10.1109/SFCS.1994.365700.

[43] M. D. Ercegovac and T. Lang, *Digital Arithmetic*. San Francisco, CA: Morgan Kaufmann, 2004.

[44] P. Christiansen, L. Binkowski, D. Ramacciotti, and S. Wilkening, "Quantum Tree Generator Improves QAOA State-of-the-Art for the Knapsack Problem," in *2025 IEEE International Conference on Quantum Computing and Engineering (QCE)*, Aug. 2025, pp. 1–10. doi: 10.1109/QCE65121.2025.00011.

[45] R. S. do Carmo, M. C. S. Santana, F. F. Fanchini, V. H. C. de Albuquerque, and J. P. Papa, "Warm-Starting QAOA with XY Mixers: A Novel Approach for Quantum-Enhanced Vehicle Routing Optimization," no. arXiv:2504.19934. arXiv, Apr. 2025. doi: 10.48550/arXiv.2504.19934.

[46] N. J. Cerf, L. K. Grover, and C. P. Williams, "Nested Quantum Search and NP-Hard Problems," *Applicable Algebra in Engineering, Communication and Computing*, vol. 10, no. 4, pp. 311–338, May 2000, doi: 10.1007/s002000050134.

[47] A. H. Land and A. G. Doig, "An Automatic Method for Solving Discrete Programming Problems," *50 Years of Integer Programming 1958-2008: From the Early Years to the State-of-the-Art*. Springer, Berlin, Heidelberg, pp. 105–132, 2010. doi: 10.1007/978-3-540-68279-0_5.

[48] T. Achterberg, "Constraint Integer Programming," Doctoral dissertation, 2007. doi: 10.14279/depositonce-1634.

[49] R. E. Bixby, "A Brief History of Linear and Mixed-Integer Programming Computation," *Documenta Mathematica Series*, vol. 6. EMS Press, pp. 107–121, Jan. 2012. doi: 10.4171/dms/6/16.

[50] G. B. Dantzig, "Discrete-Variable Extremum Problems," *Operations Research*, vol. 5, no. 2, pp. 266–288, Apr. 1957, doi: 10.1287/opre.5.2.266.

[51] R. Krishnakumar and W. Zeng, "Quantum Rounding," no. arXiv:2108.05949. arXiv, Aug. 2021. doi: 10.48550/arXiv.2108.05949.

[52] C. J. Clopper and E. S. Pearson, "The Use of Confidence or Fiducial Limits Illustrated in the Case of the Binomial," *Biometrika*, vol. 26, no. 4, pp. 404–413, 1934, doi: 10.2307/2331986.

[53] M. Thulin, "The Cost of Using Exact Confidence Intervals for a Binomial Proportion," *Electronic Journal of Statistics*, vol. 8, no. 1, pp. 817–840, Jan. 2014, doi: 10.1214/14-EJS909.

[54] T. Ichikawa *et al.*, "A Comprehensive Survey on Quantum Computer Usage: How Many Qubits Are Employed for What Purposes?," *Nature Reviews Physics*, vol. 6, no. 6, pp. 345–347, May 2024, doi: 10.1038/s42254-024-00725-0.

[55] D. Castelvecchi, "IBM Releases First-Ever 1,000-Qubit Quantum Chip," *Nature*, vol. 624, no. 7991, p. 238, Dec. 2023, doi: 10.1038/d41586-023-03854-1.

[56] Google Quantum AI *et al.*, "Suppressing Quantum Errors by Scaling a Surface Code Logical Qubit," *Nature*, vol. 614, no. 7949, pp. 676–681, Feb. 2023, doi: 10.1038/s41586-022-05434-1.

[57] S. Bravyi, O. Dial, J. M. Gambetta, D. Gil, and Z. Nazario, "The Future of Quantum Computing with Superconducting Qubits," *Journal of Applied Physics*, vol. 132, no. 16, p. 160902, Oct. 2022, doi: 10.1063/5.0082975.

[58] N. Mohseni, J.-P. Houle, I. Shehzad, G. Cortiana, C. O'Meara, and A. B. Watts, "Constrained Quantum Optimization at Utility Scale: Application to the Knapsack Problem," no. arXiv:2603.00260. arXiv, Feb. 2026. doi: 10.48550/arXiv.2603.00260.

[59] M. R. Perelshtein *et al.*, "Solving Large-Scale Linear Systems of Equations by a Quantum Hybrid Algorithm," *Annalen der Physik*, vol. 534, no. 9, p. 2200082, Sep. 2022, doi: 10.1002/andp.202200082.

[60] F. Pollmann, "Efficient Numerical Simulations Using Matrix-Product States," Sep. 2015.

[61] D. Perez-Garcia, F. Verstraete, M. M. Wolf, and J. I. Cirac, "Matrix Product State Representations," no. arXiv:quant-ph/0608197. arXiv, May 2007. doi: 10.48550/arXiv.quant-ph/0608197.

[62] C. Kim *et al.*, "ScaleQsim: Highly Scalable Quantum Circuit Simulation Framework for Exascale HPC Systems," *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 9, no. 3, pp. 62:1–62:28, Dec. 2025, doi: 10.1145/3771577.

[63] C. Cade, M. Folkertsma, I. Niesen, and J. Weggemans, "Quantifying Grover Speed-Ups beyond Asymptotic Analysis," *Quantum*, vol. 7, p. 1133, Oct. 2023, doi: 10.22331/q-2023-10-10-1133.

[64] LLC Gurobi Optimization, "Gurobi Optimizer Reference Manual."

[65] J. M. Calvin and J. Y. T. Leung, "Average-Case Analysis of a Greedy Algorithm for the 0/1 Knapsack Problem," *Operations Research Letters*, vol. 31, no. 3, pp. 202–210, May 2003, doi: 10.1016/S0167-6377(02)00222-5.

[66] S. Martello, D. Pisinger, and P. Toth, "Dynamic Programming and Strong Bounds for the 0-1 Knapsack Problem," *Management Science*, vol. 45, no. 3, pp. 414–424, Mar. 1999, doi: 10.1287/mnsc.45.3.414.

[67] J. Jooken, P. Leyman, and P. De Causmaecker, "A New Class of Hard Problem Instances for the 0–1 Knapsack Problem," *European Journal of Operational Research*, vol. 301, no. 3, pp. 841–854, Sep. 2022, doi: 10.1016/j.ejor.2021.12.009.